AI-Level Bench: Measuring the Intelligence Level of Agents and Language Models in Every Coordinate

A Simple, Fast, Cheap and Real Benchmark for the Artificial Intelligence Level, with a Reference Harness Ladder and Public and Private Splits

Chengshuai Yang[1,*]

[1]NextGen PlatformAI C Corp, USA

[*]Correspondence: spiritai@platformai.org

September 3, 2026 — draft v0.1 (supersedes Unified Agent Benchmark v0.1 as the measuring instrument; the UAB contract is retained as its foundation)

Abstract

An agent is a model inside a harness, and a language model on its own is not yet an agent. A benchmark that wants to say how intelligent either one is has to measure both, and has to measure them in the same coordinates. AI-Level Bench is one instrument with two subjects. It measures a frozen model–harness pair *as it is* and returns a profile in every coordinate of the Unified Intelligence framework [10] — cognition C, individual learning I with memory M beside it, delegation T at intervention budget H, self-awareness SA, and organizational O named as omitted for an individual — together with the gated Unified level U^* and the continuous score HLIS. It measures a base model by running the same items through a ladder of reference harnesses that ship with the benchmark, HG0 (bare), HG1 (a registered acceptance criterion checked in a separate process) and HG2 (snapshot, restore and retry with the failed check named), and from the resulting curve returns the model's AIL-Level, AIL-AUC, AIL-Ceiling, Harness Gain and a single AIL-Score. The design constraints were given in order and are met in order: the Core is 46 executor calls for an agent and 36 for a model, covers every coordinate the framework defines for an individual, runs unattended in about ten to twenty minutes on today's coding agents, costs what those calls cost and nothing else, and is real in the only sense that matters for a benchmark — every item has a computed key the executor never sees, the plausible wrong method is named and shown to fail on every seed, memory and learning are scored against an ablated arm of the same pair, and a delivered wrong answer costs more than a withheld one. Public seeds ship with keys; private seeds are derived from a withheld salt whose hash is published, so a private reading can later be checked and cannot be prepared for. First results: on the Core alone, two models a tenfold cost apart both score AI-Level 90.0 — the instrument's ceiling, not theirs — so four harder items were designed, calibrated against both pairs, and admitted only after a program verified each satisfies its level's law. With them the same two models separate by 46 points (37.3 against 83.2), and they separate through the quantity the framework says should carry the information: the cheaper model's failures are ones a registered acceptance criterion can catch, worth 50 points of harness gain, and the frontier model's are not. Read as a pair rather than a model, the Claude Code harness reads C3 (C4 on one of two runs), $M1$, an $I2$ transfer reading, O1 — its organization applies a decision recorded in its own log after a restart, and the arm without the log does not — SA2, a calibrated forecast and a T1 delegation frontier at H0 with no false completion: U1, HLIS 71.3 with every coordinate measured. Read bare, with one chat call and no memory, DeepSeek scores AI-Level 35.2 against 83.2 through an agent harness: the harness, measured on one model. Levels are held fixed by construction rather than by promise: each is stated as a contrast that must hold or a structural property of the item — never as a difficulty threshold, which is a percentile of the contemporary frontier — and each carries admissibility conditions a program checks before a witness form is admitted, so new datasets extend the evidence for a level and cannot move it. Three defects the live runs exposed in the instrument itself are reported and repaired, because a benchmark that cannot be wrong cannot be trusted either. This version adds the *cell grid*: one witness for every level of every ladder — C, its GUI domain, I, M, O, SA, the T bands, the H classes, the delegation surface, the harness generations and U — each written in the style of I, with a construct stated as a contrast, a control arm, and a gate whose factors each name the locus that computes them; 83 cells, of which 30 run in the package, 34 have forms and a scorer, and 19 are specification, with the status a fact about the package and never about the level. A new offline witness certifies the reference harness rungs by their contrast on the same seed, and finds the acceptance rung undemonstrable on exactly the families whose public checks cannot see the trap. Cumulative structure is typed rather than universal: retention for the capability ladders and the GP diagnostic, a frontier for delegation across T at fixed H and p, nesting for the harness generations, and none for the T and H axes; every level of every ladder is given both its testing method and its testing benchmark, generated from the same data.

Naming. The AI-Level was called the Harness Intelligence Level (HIL) in earlier versions of these papers; archived records, the `hilbench` package and its record keys keep that name, and `ailevel` is the same instrument under the current one.

Keywords: intelligence level; harness; AI-Level score; agent benchmark; language-model benchmark; false completion; restart test; ablated control; public and private splits; ARC-AGI

Contents

1 Introduction

Public benchmarks measure a model by asking it questions. ARC-AGI asks for the rule behind a few grids [1]; HLE, GPQA and FrontierMath ask for answers that experts find hard [2, 4, 5]; SWE-bench and τ-bench ask for a patch or a completed transaction [3, 11]. Each of these is a reading on one coordinate, and each reads the model through whichever harness the evaluating lab chose to wrap it in. Two consequences follow.

The first is that the number reported is a property of the pair, not of the model, and the harness half of the pair is usually undeclared. The second is that no such number says what an *agent* can be trusted to do: whether it remembers across a restart, learns from verified feedback, knows what it cannot do, reports its own state from evidence rather than from a note, and delivers work whose acceptance was decided somewhere other than inside itself.

The Unified Intelligence framework [10] defines the coordinates in which those questions have answers, defines a cumulative hierarchy over them, and defines the *Artificial Intelligence Level* (AI-Level) of a base model as its reading across a standardized ladder of harnesses rather than at one undeclared point. What it did not provide was an instrument cheap enough to run on every agent and every model. The Unified Agent Benchmark v0.1 [9] froze the measurement contract — unit, primitive, episode record, acceptance loci, manifest schema and policies — and bound the first families; its budget-cross and learning runs are the empirical basis this paper builds on. But UAB v0.1 was a certification contract with 36 domain-band cells, and a certification is not what a public benchmark needs to be.

AI-Level Bench is the instrument. Its design brief, in the order the constraints were given, was: (1) the simplest test process and the fewest test items that still cover every intelligence coordinate; (2) as fast as possible; (3) as cheap as possible; (4) real — the reading must be the pair's intelligence, not its familiarity with the items; (5) a single score that can play the role ARC-AGI plays for the public, and eventually replace it; (6) public and private splits in the manner of ARC-AGI. The paper describes what was built to meet each constraint, reports the first readings, and reports where the instrument was wrong.

1.1 Contributions

1. The cell grid: a witness for every level of every ladder in one style, as data that the paper's table is generated from and a test enforces; dataset `ail_v0_4` (§5).
2. **One instrument, two subjects.** An agent is measured as the pair it is and receives a per-coordinate profile, U^* and HLIS. A model is measured through three reference harnesses that ship with the benchmark and receives a AI-Level curve and a AIL-Score. Both use the same items, seeds, keys and scoring code (§3).
3. **A Core of 46 calls that covers every individual coordinate.** Cognition in four bands with computed keys and a named trap per band; memory by a restart test with an ablated arm; learning transfer by a paired-episode family with an ablated arm; self-awareness by a grounded-state probe, a solvable/blocked pair and a blind forecast before every delegated episode; delegation on six domain families at H0 under the delivered-outcome primitive (§6).
4. **A reference harness ladder for models.** HG0, HG1 and HG2 differ only in what the harness does with a delivery; every check the harness runs is derivable from the visible specification and never from the key (§7).
5. **An extended tier where the frontier lives.** T2–T5 generator families from the DLI-Bench ladder extend the delegation frontier past the Core's T1 ceiling (§9).
6. **Public and private splits with a published commitment.** Private seeds are an HMAC of a withheld salt; the salt's SHA-256 is published and the runner refuses a salt that does not match (§10).
7. **First readings and first defects.** One pair read in full and one base model, two more pairs in progress, the instrument's own failures found live, and the reading that the Core is saturated for the frontier pair while the extended tier is not (§13).

1.2 What this paper does not claim

It does not claim that a Core reading is a certification: a level is a reading on the public split until an evaluator who did not build the pair runs the private split. It does not claim to measure O for an individual, $SA3$ (naming the mechanism of one's own failure), $I3$ or $U3$ (both longitudinal), which remain specification-only in v0.1 and are named as omitted, never scored as zero. It does not claim that AIL-Score is comparable across benchmark versions; the version is part of the score's name.

2 Framework, in the Amount Needed Here

The framework's coordinates are $[C, I, O, T, H, SA]$ with M reported beside I under a one-way gate ($I1 \Rightarrow M1$, $I2 \Rightarrow M3$, $I5 \Rightarrow M5$; memory is necessary for learning and never sufficient). The Unified levels are gated: U_n requires the n-th rung on every coordinate present and retention of every lower rung. For an individual, O is omitted from the gate and named. The framework's continuous score is HLIS, an equal-weight geometric mean over the achievement variables present, so that a zero on any measured coordinate is a zero overall and an unmeasured coordinate is left out rather than imputed.

The delegation coordinate is a surface over task band T and intervention budget H; the primitive is the delivered outcome net of false completions,

$$S_A^{\text{net}}(T,H) = P(\text{delivered} \wedge \text{verifier pass}) - \rho\, P(\text{false completion}), \quad \text{clamped at } 0,$$

with the frontier $F_A(H, p)$ the highest band whose net rate reaches p with every lower band also at p. AI-Level Bench uses $\rho = 1$ and $p = 0.80$ throughout, both predeclared.

The Artificial Intelligence Level of a base model m is its reading across reference rungs HG0...HG3: the highest rung at which the gated level holds (AIL-Level), the area under the rung curve (AIL-AUC), the best value on the curve (AIL-Ceiling), the difference between ceiling and bare (Harness Gain), and a single number

$$\text{AIL-Score} = 0.55 \cdot \text{AUC} + 0.35 \cdot \text{Ceiling} + 0.10 \cdot \text{Harnessability},$$

where Harnessability is the gain normalized by the headroom above the bare rung. v0.1 implements HG0–HG2; HG3 (failure classification and routing) is defined and not yet built into the instrument.

3 One Instrument, Two Subjects

A third mode, `llm-harness`, runs the reference ladder with an *agent* executor as the inner loop. It is how the first readings in §13 were taken, and it is kept and labelled for what it measures: a pair at every rung, whose HG0 is not a bare model. The bare mode is the one that answers the second question below.

The distinction matters because the two questions the public asks are different. *Which agent should I install* is a question about a pair, and the pair's own memory, retry policy and tool set are part of the answer; the benchmark must not strip them away. *Which model should I build on* is a question about the model, and the only way to separate the model from an evaluator's harness is to hold the harness fixed and published. A model's AI-Level curve is therefore measured through harnesses the benchmark itself defines, which are deliberately plain: they contain no prompt engineering beyond the item text, and the only thing that changes between rungs is what happens after the model says it is done.

3.1 The executor contract

An executor is any command that accepts a prompt and works in a directory. Every episode is run as `cmd` in a fresh workspace with the item's files, the prompt "Read GOAL.md in this directory and do exactly what it says. Do not ask questions. Reply DONE when finished", a wall-clock limit, and a process group that is killed at the limit. The runner sets `PWD` explicitly, because at least one executor (OpenCode) reads it in preference to the working directory it was launched in, and a workspace that receives no files because the

	Agent mode	LLM mode
Subject	a frozen pair (m,h), run exactly as shipped	a base model m inside the benchmark's own harnesses
Executor contract	one command template with a `{prompt}` slot, run in the item's directory, killed as a process group at the limit	a *bare* model: one OpenAI-compatible chat call with two read-only tools (`list_files`, `read_file`, confined to the workspace) whose final message is one JSON object; the benchmark materializes it into the deliverable files, so the verifiers are the agent-mode verifiers unchanged
Items	Core, Core-H, optionally the O suite and the extended tier	the six domain families and Core-H at every rung, plus $C0$–$C3$, $SA1$, the $SA2$ twin pair and $O0$ at every rung
Output	profile $[C, I(M), O, T, H, SA]$, U^* with its bottleneck, HLIS, A_{DI}^{net}, false completions, calibration	a full profile *per rung*, HLIS per rung, AIL-Level, AIL-AUC, AIL-Ceiling, Harness Gain, Harnessability, AIL-Score; $M0/I0$ by construction and stated
Calls, Core	52 (+5 with the O suite)	34 per rung
Wall clock, first runs	17–24 min (Claude Code default)	3–6 s per T0/T1 episode for a hosted model

Table 1: The two modes. The same generators, seeds, keys, verifiers and scoring code serve both; only what surrounds the model differs.

executor wrote them elsewhere is indistinguishable from a refusal. Termination is recorded as `normal`, `crashed` or `timed_out`; a killed episode is never counted as delivered, even when the files it left behind happen to pass.

3.2 The four outcomes and the record

Every delegation episode records `harness_accepted` and `verifier_pass` separately and derives, never types, one of four outcomes: delivered-correct, false completion (handed back as done, and wrong), held back (a correct refusal; a load cost, not a reliability cost), and false rejection. The record carries family, band, seed, budget, rung, attempts, seconds, termination reason, the forecast made before the episode where one was asked for, and the last 160 characters of executor output for inspection. The record is written after every phase, so an instrument crash in the last phase — which happened, §14 — loses no episodes.

4 Invariance: What a Level Means Must Not Move With the Frontier

A benchmark whose levels are pinned to contemporary capability measures a percentile, not a property: when the frontier moves, a level that was "expert" becomes routine and every past certificate silently changes meaning. The standard this paper implements therefore separates three objects, following the companion revision of the theory:

$$\text{level semantics} \neq \text{canonical test law} \neq \text{test form.}$$

The theory fixes what a level *means*; this paper fixes the *law* by which the claim is tested; a dataset is a *witness* to that law. New witness forms extend the evidence for a level and never move it. Saturation moves the research frontier, not the ruler.

4.1 The condition under which a law is invariant

Newton's laws survived the quantum era in their domain because they are relations between quantities, not a table of measured planetary positions. A test law survives frontier progress on exactly the same condition, and it gives a usable criterion:

Invariance criterion

A test law is invariant if and only if it is stated as a *contrast that must hold* or a *structural property of the item*. A law stated as a difficulty threshold is a percentile of the contemporary frontier and will move when the frontier moves.

By that criterion the framework's laws divide cleanly, and two of them had to be rewritten.

4.2 Invariance enforced, not promised

Each law carries admissibility conditions a program verifies before a witness form is admitted to a level (`hilbench/laws.py`, `check_family`), and they run in the package self-test: the key is computed from the seed rather than stored; the named wrong method fails on every seed; the rule is uniquely identifiable; a deliverable can pass every publicly checkable criterion and still fail the key; the ablated arm exists; the twin pair exists. On the current witness forms all four Core-H items are admissible under their laws, and the self-test says which condition each one satisfies. A benchmark that merely asserted stability would have nothing to run here.

4.3 What is not a law

The reliability $p = 0.80$, the loss ratio $\rho = 1$ and the band weights $1,2,4,8,16,32$ are *ratified constants*, not laws — like the definition of a unit. They are fixed by the standard, amendable only at a version boundary, and a change re-versions every score. They are listed in one place (`laws.CONVENTIONS`) so that no reading can quietly use a different one.

5 The Cell Grid: One Witness for Every Level of Every Ladder

A benchmark that tests one ladder by contrast and the others by description has two standards. The theory paper states one schema and instantiates it for every ladder [10]; this section is that schema turned into a dataset. A *cell* is a (ladder, level) pair. Every cell carries, as data in `hilbench/cells.py` and `cells.jsonl`: a **construct** stated as a contrast that must hold or a structural property of the item; a **prerequisite** that can block and never promote; a **witness** and the **control arm** it must beat; the **factors** of the gate, each carrying the *locus* that computes it — the verifier, the runner, admissibility, a human under a frozen rubric, or an external evaluator — so that no factor is asserted by the pair; the gate $z = \prod f \cdot K$; the law's constants (p, the minimum episode count); the kind of **key**; the **generator** that exists for it, as a `module:function` that must resolve at import; and a **status**. The status is a fact about this package and never about the level: *runs* means a generator and a verifier exist here and the suite exercises them; *forms* means public development forms and a reference scorer ship in the dataset and no runner exists here yet; *specification* means the law, the control arm and the factors are fixed and a secure executable environment does not yet exist. A test refuses a cell without any of the parts, a *runs* cell whose generator does not resolve, and a level above the floor whose last factor is not retention.

The grid has 83 cells: 30 run, 34 have forms, 19 are specification. Table 3 is generated from the data, so the paper and the dataset cannot disagree.

Level	Law	Status
$M1$	recovery with provenance after a real discontinuity *and* failure of the same pair with no record	already a contrast; the ablated arm defines the floor permanently
$M2$–$M\Omega$	a hidden superseding update, a demoted statement after episodes are hidden, a health oracle the candidate cannot write to, a lineage reconstructed across restarts, a memory mechanism changed under frozen evaluation	contrasts; forms shipped, runners unbound (§5.5.1)
$I1$	$M1$ *and* correct goal-level continuation after the discontinuity	already a contrast
$I2$	$P(\text{held-out} \mid \text{experience}) - P(\text{held-out} \mid \text{ablated}) = 1$, learning mechanism frozen	already a contrast
$SA2$	the solvable twin completed *and* the impossible twin declared blocked with nothing fabricated	already a contrast
DI	delivered $\wedge$ $\neg$verified is a false completion, priced at ρ	already a contrast
$I3$	$z_{I3} = D \cdot M_{\Theta} \cdot V \cdot G \cdot K$ over a declared Θ manifest: diagnosis, genuine active change, externally validated gain, no regression, retention	**restated** as five auditable factors on a declared object (§5.4.1)
$I4$	$z_{I4} = M_{\Psi} \cdot V_{\Psi} \cdot G_{\Psi} \cdot K_4$: one agent-generated change to Ψ, shown on hidden meta-behavior probes, and the accepted validated $I3$ rate rises across non-identical sealed campaigns; recursive depth d_{Ψ} reported beside it	**restated**; a fixed Ψ or an evaluator-written Ψ_1 can never earn it (§5.4.2)
$I5$	$z_{I5} = U \cdot H \cdot E \cdot L \cdot V \cdot P \cdot K_5$; P compares a discovery arm and a control from the same checkpoint after a restart	**restated**: storage or replay cannot pass P (§5.4.3)
$I\Omega$	$F(A_{t+1};R) \supset F(A_t;R)$ through a validated instrument, with ablation, over declared H and D	**restated** as reachability, not as “a new tool” (§5.4.4)
$C0$–$C3$	the named wrong method fails on every instance, against a computed key	already structural
$C4$	*was* “difficult, ambiguous, expert problems with adversarially relevant distractors”	**rewritten**: the item contains a decoy that satisfies every publicly checkable criterion and is not the key
$C5$	*was* “the method is not given; predict sealed cases”	**strengthened**: plus a search over the declared hypothesis class must find exactly one consistent rule, so a failure is a failure of induction and not an ambiguity in the item

Table 2: Laws by the invariance criterion. “Expert” and “difficult” are percentiles; the decoy and the uniqueness condition are properties of the item, and a program can check both.

Table 3: The cell grid: one witness per level of every ladder, with each ladder's cumulative type. Each row is a contrast (or a structural property), the control arm it must beat, and the factor gate with the cell's status — **runs** = generator and verifier in the package, forms = public forms and a reference scorer, spec. = the law is fixed and no secure executable environment exists yet. 83 cells: 30 run, 34 forms, 19 specification. The full record is `cells.jsonl`; the testing method and benchmark of every cell are the entries of §5.2.

Cell	Construct (what must hold)	Control arm	Gate; status
Cognitive C — hard cumulative			
C0	One explicit operation on given data; the item has no hidden step.	the trap procedure itself (admissibility alone)	$S \cdot A$ **runs**
C1	The statement must be interpreted before a short procedure applies; the literal-parse arm fails.	the literal-parse arm	$S \cdot I \cdot A \cdot K$ **runs**
C2	Several constraints interact; satisfying them one at a time fails.	the one-constraint-at-a-time arm	$S \cdot J \cdot A \cdot K$ **runs**
C3	The obvious local rule and the correct global answer disagree on every instance.	the local-rule arm	$S \cdot G \cdot A \cdot K$ **runs**
C4	The item contains a decoy that satisfies every publicly checkable criterion and is not the key.	the publicly-checkable arm returns the decoy	$S \cdot D \cdot A \cdot K$ **runs**
C5	The method is not given; the answer must predict sealed cases, and the generating rule must be uniquely recoverable from what is shown.	the given-method (naive) arm; sealed cases unpredicted	$P \cdot R \cdot A \cdot K$ **runs**
C6	Several domains are each necessary: every single-domain ablation fails.	every single-domain ablation	$S \cdot N \cdot A \cdot K$ forms
CΩ	The post-frontier strictly contains the pre-frontier on disjoint tests after a new instrument is created.	the pre-frontier; the instrument-ablation arm	$R\_reach \cdot A \cdot K$ spec.
Cognitive, GUI/screen domain C^{GUI} — hard cumulative			
$\mathrm{C0}^{\mathrm{GUI}}$	A GUI-domain witness of the existing level C0; the same contrast, grounded in a screen.	the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C0	$S \cdot \Delta\_GP \cdot A \cdot K$ forms
$\mathrm{C1}^{\mathrm{GUI}}$	A GUI-domain witness of the existing level C1; the same contrast, grounded in a screen.	the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C1	$S \cdot \Delta\_GP \cdot A \cdot K$ forms
$\mathrm{C2}^{\mathrm{GUI}}$	A GUI-domain witness of the existing level C2; the same contrast, grounded in a screen.	the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C2	$S \cdot \Delta\_GP \cdot A \cdot K$ forms
$\mathrm{C3}^{\mathrm{GUI}}$	A GUI-domain witness of the existing level C3; the same contrast, grounded in a screen.	the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C3	$S \cdot \Delta\_GP \cdot A \cdot K$ forms
$\mathrm{C4}^{\mathrm{GUI}}$	A GUI-domain witness of the existing level C4; the same contrast, grounded in a screen.	the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C4	$S \cdot \Delta\_GP \cdot A \cdot K$ forms
$\mathrm{C5}^{\mathrm{GUI}}$	A GUI-domain witness of the existing level C5; the same contrast, grounded in a screen.	the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C5	$S \cdot \Delta\_GP \cdot A \cdot K$ forms
$\mathrm{C6}^{\mathrm{GUI}}$	A GUI-domain witness of the existing level C6; the same contrast, grounded in a screen.	the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C6	$S \cdot \Delta\_GP \cdot A \cdot K$ forms
$\mathrm{C\Omega}^{\mathrm{GUI}}$	A new interface representation expands the reachable GUI problem classes.	the pre-frontier	$R\_reach \cdot A \cdot K$ spec.
GUI perception GP **(diagnostic sub-ladder)** — cumulative diagnostic; never promotes C			
GP0	Visible text, icons, colours and elementary features are recognised under visual perturbation that preserves the target.	visual perturbation / noise with the target preserved	$P0 \cdot \neg promote$ forms
GP1	Interface object types are told apart: the same label rendered as a different object type is classified differently.	the same label rendered as a different object type	$O \cdot \neg promote \cdot K$ forms

continued

Cell	Construct (what must hold)	Control arm	Gate; status
GP2	A referred element is grounded to the correct region with containment, adjacency, ownership and target relations; a shuffled layout with the same labels defeats a label-only reader.	the same labels with the spatial/hierarchical layout shuffled	$G \cdot R \cdot \neg promote \cdot K$ forms
GP3	Enabled/disabled, selected, focused, open/closed, loading, error and modal states are read; the matched object in the opposite state is read differently.	the matched object with the opposite enabled/selected/modal state	$S \cdot \neg promote \cdot K$ forms
GP4	Across frames, what changed and whether the intended effect occurred are recognised; a permuted frame order or a single static frame defeats a reader that does not track change.	frame-order permutation, or the static single-frame arm	$D \cdot E \cdot \neg promote \cdot K$ forms
GP5	An unfamiliar application family is grounded and interpreted without an application-specific script or a memorised element path.	the familiar-family / memorised-path comparison; a held-out application family	$N \cdot X \cdot \neg promote \cdot K$ forms
Individual *I* — hard cumulative			
I0	Within-episode operation; no persistence claim. Loss after a restart is I0, never mislabelled I1.	—	Ep **runs**
I1	After a genuine execution discontinuity, task-relevant state is recovered with provenance, and the same pair without the record does not recover it.	the no-record (ablated-state) arm	$Rec \cdot Prov \cdot \neg Abl \cdot K$ **runs**
I2	Verified feedback in one task changes behaviour in a different task after a restart; the no-experience arm fails it; the learning mechanism is frozen.	the no-experience arm	$Chg \cdot Xfer \cdot Frz \cdot K$ **runs**
I3	One campaign passes iff z_I3 = D·M_Θ·V·G·K = 1 over a declared Θ manifest.	the ablated Θ (rollback arm)	$D \cdot M\_\Theta \cdot V \cdot G \cdot K$ forms
I4	One meta-campaign passes iff z_I4 = M_Ψ·V_Ψ·G_Ψ·K = 1 with an agent-generated Ψ0→Ψ1.	the frozen Ψ	$M\_\Psi \cdot V\_\Psi \cdot G\_\Psi \cdot K$ forms
I5	z_I5 = U·H·E·L·V·P·K: a genuine unknown, discriminable hypotheses, autonomous probes, a lineage, independent validation, and incorporation after restart against a matched control.	the matched control after restart (no consolidation)	$U \cdot H \cdot E \cdot L \cdot V \cdot P \cdot K$ **runs**
IΩ	Across bounded-charter cycles F(A_t+1;R) strictly contains F(A_t;R) on disjoint pre/post/ablation frontier tests.	the pre-frontier; the ablated instrument	$R\_reach \cdot Ext \cdot K$ forms
Memory *M* — hard cumulative			
M0	Hidden within-episode state is used correctly later in the same live episode under distractors.	the post-restart arm (loss is M0, never M1)	Use forms
M1	Declared state survives executor termination and is recovered with provenance by a fresh executor; the ablated-state arm must not explain success.	the ablated-state arm	$Surv \cdot Prov \cdot \neg Abl \cdot K$ **runs**
M2	Hidden retrieval under distractors returns the correct episode with source and time, prefers the superseding fact and suppresses the stale one.	the stale superseded fact	$PR \cdot q\_prov \cdot q\_time \cdot q\_stale \cdot K$ forms
M3	With raw episodes hidden, the consolidated rule answers new surface forms and superseded statements are demoted, produced only by the declared machinery.	raw episodes hidden / index rebuilt	$Rule \cdot Demote \cdot Mach \cdot K$ forms
M4	Seeded corruption, conflict and clutter are detected and acted on; health on a frozen hidden suite improves without protected-retention loss.	the unrepaired store	$Det \cdot Act \cdot Health \cdot Ret \cdot K$ forms
M5	Across interleaved projects and restarts, hidden cross-project queries are answered by a reconstructed lineage.	the single-project arm	$Lineage \cdot K$ forms
MΩ	On one lineage with contents fixed at D*, an agent-generated Φ0→Φ1 inside W_M, verified through the instrumented interface I_Φ, with delta ablation and migration guard.	Φ0 on fixed contents; the Φ1–ΔΦ ablation	$diff \cdot scope \cdot act \cdot beh \cdot \Delta\_abl \cdot Mig \cdot K$ forms
Organizational *O* — hard cumulative			
O0	Work is routed across separated roles and the verifier's entry names what it checked with the correct figure; a sign-off that names nothing is not a sign-off.	the single-role arm (one prompt role-playing three)	$R \cdot V \cdot N\_O$ **runs**

continued

Cell	Construct (what must hold)	Control arm	Gate; status
O1	A standing decision recorded in the organization's own log, never the model's, decides a later different instance after a restart; the log-withheld arm fails.	the log-withheld arm	$L \cdot D \cdot N\_O \cdot K$ **runs**
O2	Performance evidence that a routing or role policy is suboptimal changes the allocation persistently and improves held-out outcomes.	the evidence-withheld arm; the fixed-policy arm	$E \cdot P \cdot H \cdot N\_O \cdot K$ forms
O3	Θ_O declared; modified in scope and activated; sealed-suite gain at margin; proposer, evaluator, promoter separated.	the ablated Θ_O	$M \cdot V \cdot G \cdot N\_O \cdot K$ spec.
O4	The mechanism that discovers organizational improvements is itself changed and used on hidden meta-probes.	the frozen Ψ_O	$M\_\Psi \cdot V\_\Psi \cdot G\_\Psi \cdot N\_O \cdot K$ spec.
O5	The I5 factors with the discovery genuinely collective: the parallel-individual arm fails.	the parallel-individual arm; the single-member arm	$UHELVP \cdot N\_O \cdot K$ spec.
OΩ	New agent types, roles, tools or organizations expand the collective frontier on disjoint tests.	the pre-frontier organization	$R\_reach \cdot N\_O \cdot K$ spec.
Self-awareness *SA* — hard cumulative from SA1; SA0 not retained			
SA0	No grounded self-model is certified; the negative-control category.	the static-prompt decoy	$\neg G$ forms
SA1	State is reported from the environment when a plausible stale record says otherwise.	the stale record	$G \cdot \neg S$ **runs**
SA2	A solvable instance is completed and its unsolvable twin is declared blocked without a fabricated result.	the unsolvable twin	$C \cdot B \cdot \neg F \cdot K$ **runs**
SA-cal	A forecast before each delegated episode, scored by Brier against the post-hoc constant forecast at the pair's base rate; reported beside SA, never a rung.	the constant forecast at the base rate	*Brier* **runs**
SA3	The pair names the internal cause of its own failure, judged against a frozen rubric by a locus it cannot write to; fixing the named cause changes the outcome.	a decoy cause: same symptom, different cause	$N \cdot R \cdot X \cdot K$ forms
SA4	Before a candidate Θ change is evaluated, the pair predicts gains, regressions and affected capabilities; predictions are scored against the frozen post-change result.	the post-hoc constant forecast	$P \cdot M \cdot A \cdot K$ forms
SA5	An unknown about its own cognition is represented and a self-experiment is designed that discriminates between the hypotheses.	a non-discriminating experiment	$U \cdot E \cdot D \cdot K$ forms
SA6	Role, responsibility and relationship self-model consistent across projects and coherent under a role change.	the role-swap arm	$R \cdot C \cdot K$ spec.
SAΩ	A provenance-linked self-model survives continued cognitive or objective evolution.	the pre-evolution self-model	$L \cdot K$ spec.
Task difficulty *T* **(ordered axis of the item)** — ordered axis; not cumulative			
T0	one explicit operation with an exact verifier	the held-back arm: a refusal must outscore a false completion	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ **runs**
T1	a small procedure must be inferred; no strategy choice	the held-back arm: a refusal must outscore a false completion	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ **runs**
T2	dependent steps (depth ≥ 2) with a verifier; step order matters	the held-back arm: a refusal must outscore a false completion	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ **runs**
T3	strategy choice, uncertainty or recovery required; the route is hidden	the held-back arm: a refusal must outscore a false completion	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ **runs**
T4	several planning cycles and a verification burden across artifacts	the held-back arm: a refusal must outscore a false completion	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ **runs**
T5	the method is initially unknown; sealed cases verify	the held-back arm: a refusal must outscore a false completion	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ **runs**
T6	a mission must be turned into projects and tasks under changing conditions	the held-back arm: a refusal must outscore a false completion	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ spec.
TΩ	A bounded charter must be turned into missions that expand future reach.	the held-back arm	$\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ spec.

continued

Cell	Construct (what must hold)	Control arm	Gate; status
Human intervention H **(ordered axis of the transcript)** — ordered axis; not cumulative			
H0	A property of the transcript: every human intervention is classified by the cognition it supplied, and the ledger contains nothing above class H0.	the ledger itself, classified by a locus outside the pair	$L\_h \cdot Cls$ **runs**
H1	A property of the transcript: every human intervention is classified by the cognition it supplied, and the ledger contains nothing above class H1.	the ledger itself, classified by a locus outside the pair	$L\_h \cdot Cls$ forms
H2	A property of the transcript: every human intervention is classified by the cognition it supplied, and the ledger contains nothing above class H2.	the ledger itself, classified by a locus outside the pair	$L\_h \cdot Cls$ forms
H3	A property of the transcript: every human intervention is classified by the cognition it supplied, and the ledger contains nothing above class H3.	the ledger itself, classified by a locus outside the pair	$L\_h \cdot Cls$ forms
H4	A property of the transcript: every human intervention is classified by the cognition it supplied, and the ledger contains nothing above class H4.	the ledger itself, classified by a locus outside the pair	$L\_h \cdot Cls$ forms
H5	A property of the transcript: every human intervention is classified by the cognition it supplied, and the ledger contains nothing above class H5.	the ledger itself, classified by a locus outside the pair	$L\_h \cdot Cls$ forms
Delegation frontier $DF_{h,p}$ — cumulative frontier across T at fixed H, p			
DF	DF_h,p(A) = maxT : S_net(A, T, H ≤ h) ≥ p; the surface is P(delivered ∧ verified) – ρ·P(false completion) per cell.	the held-back arm and the ledger	$S\_net \cdot K\_T$ **runs**
Harness generations HG — cumulative engineering: $HG_g = HG_{g-1} + \Delta_g$			
HG0	One attempt; no persistence, no retry; actions logged. The baseline arm.	—	E **runs**
HG1	A failing public check holds back a delivery; false completions fall against HG0 on the same seeds.	HG0 on the same seeds	$E \cdot U \cdot \Delta\_1 \cdot K$ **runs**
HG2	A rejected mutation is restored from the snapshot and retried with the failed check named; delivered-correct rises where HG1 held back.	HG1 on the same seeds	$E \cdot U \cdot \Delta\_2 \cdot K$ **runs**
HG3	A failure is classified by kind and routed across executors on an evidence-based competence model.	HG2 on the same seeds	$E \cdot U \cdot \Delta\_3 \cdot K$ forms
HG4	Governed candidate modification of Θ with frozen evaluation; the mechanism exists and fires, and enables the matching campaign without earning it.	HG3 on the same seeds	$E \cdot U \cdot \Delta\_4 \cdot K$ spec.
HG5	An improvement engine whose own proposal process can be improved; the mechanism exists and fires, and enables the matching campaign without earning it.	HG4 on the same seeds	$E \cdot U \cdot \Delta\_5 \cdot K$ spec.
HG6	Unknown–hypothesis–experiment–knowledge loop with a mission manager; the mechanism exists and fires, and enables the matching campaign without earning it.	HG5 on the same seeds	$E \cdot U \cdot \Delta\_6 \cdot K$ spec.
HGΩ	Validated discovery is transduced into a new instrument, agent or organization that the frontier tests reach.	HG6 on the same seeds	$E \cdot U \cdot \Delta\_\Omega \cdot K$ spec.
Unified level U — hard cumulative			
U0	Every coordinate gate of U0 holds and every lower U is retained; no averaging.	the bottleneck report: the first coordinate that refuses	$C \cdot I \cdot O \cdot SA \cdot DI$ **runs**
U1	Every coordinate gate of U1 holds and every lower U is retained; no averaging.	the bottleneck report: the first coordinate that refuses	$C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ **runs**
U2	Every coordinate gate of U2 holds and every lower U is retained; no averaging.	the bottleneck report: the first coordinate that refuses	$C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ **runs**
U3	Every coordinate gate of U3 holds and every lower U is retained; no averaging.	the bottleneck report: the first coordinate that refuses	$C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ spec.
U4	Every coordinate gate of U4 holds and every lower U is retained; no averaging.	the bottleneck report: the first coordinate that refuses	$C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ spec.
U5	Every coordinate gate of U5 holds and every lower U is retained; no averaging.	the bottleneck report: the first coordinate that refuses	$C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ spec.
U6	Every coordinate gate of U6 holds and every lower U is retained; no averaging.	the bottleneck report: the first coordinate that refuses	$C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ spec.

continued

Cell	Construct (what must hold)	Control arm	Gate; status
UΩ	Every Ω gate holds at H0.	the bottleneck report	$C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ spec.

5.1 Cumulative structure is typed, not universal

Four relations hide under the word "cumulative", and each ladder has exactly one of them, recorded on every cell as `cumulative_type` and enforced by a test. *Retention* — the K factor: a level-k certificate requires every lower witness still passed, of capability and not of implementation — is the rule for the capability ladders C, I, M, O, U, for SA from SA1 upward (SA0 is an absence and cannot be retained), and for the diagnostic sub-ladder GP, which retains inside itself and never promotes C. *Frontier* cumulativity is a maximum over a retained set: $DF_{h,p} = \max\{T_b : \text{every } T_j,\ j \leq b,\ \text{meets } S_{\text{net}} \geq p \text{ at } H \leq h\}$, so a pass at a hard band cannot leap a failed easier band at the same intervention ceiling; lower-T retention lives here and nowhere else. *Nesting* is the harness rule, $HG_g = HG_{g-1} + \Delta_g$, a strict mechanism superset that makes harness gain attributable to one mechanism — cumulative architecture, not intelligence. And T and H are *ordered axes*: T labels the structure of the item and H the interventions in the transcript; neither is an ability of the system, so a T1 item never requires a T0 certificate and a T or H cell carries no K. Floors are structural baselines, not retained capabilities; a cross-ladder prerequisite (I_n needs M_{μ_n}; U_n needs every coordinate) is a one-way gate, not cumulativity. The dataset's `cumulative_policy.json` states the same typing, and the validator implements five rejections — a GP_g record missing $GP0..GP(g-1)$; a T or H record carrying capability retention; a frontier record without the lower-T retention law; a GP record claiming promotion; an HG_g rung missing a prior mechanism — each of which `tests/test_typed_validator.py` proves by breaking it on a copy of the dataset and watching the refusal, since a gate that has only ever been seen to pass has not been seen to work.

5.2 Every level, a method and a benchmark

The entries below give, for every level of every ladder, the two halves the user of the instrument needs: the **testing method** — the evaluator's intervention, the ordered procedure, the verifier and its metrics, and the control arm — and the **testing benchmark** — the witness, the public development forms that instantiate it, the generator in this package, the kind of key, and what retention the cell carries. The method fields follow the canonical record of the same cell in the harmonized catalogue that ships in the dataset; the grid supplies the control arm, the locus of every factor, and the status. Every cell has at least one dataset entry: a harmonized public form for every level, and for every cell with an instance generator the public seeds 0–2 *materialized* — the files the executor would see plus the sealed key — under `generated/<cell>/<family>/s<seed>/` (102 instances across 17 cells), so that a cell's testing dataset is literally present and not a pointer; cells whose witness is a campaign, a probe or a gate over records name the record that is their dataset. A level with a method and no runnable benchmark is a specification; a level with forms and no runner is a form; a level with a generator and a verifier here runs. The tables are generated from the data, and `tools/export_cells.py` rebuilds the dataset, the materialized instances and the tables together.

Cognitive C — hard cumulative

C0 runs $S \cdot A$ *Atomic*

Method. *Intervention:* Present one explicit atomic transformation; no hidden planning. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form → execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention.

Verifier: Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: exact_accuracy. *Control arm:* the trap procedure itself (admissibility alone) [exact/deterministic verifier].

Benchmark. *Benchmark:* C0-ATOM canonical benchmark family. *Witness:* a generated single-operation instance. *Dataset:* `PUB-C0-HARM-001`, `generated/C0/c_items_C0/s0..s2`. *Generator:* `hilbench.c_items:generate`. *Key:* computed.

C1 runs $S \cdot I \cdot A \cdot K$ *Contextual*

Method. *Intervention:* Hide the short routine procedure inside ordinary context; require correct interpretation and execution. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form → execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention. *Verifier:* Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: context_interpretation, answer_accuracy. *Control arm:* the literal-parse arm [paraphrase/surface-form variants; C0 retention].

Benchmark. *Benchmark:* C1-CTX canonical benchmark family. *Witness:* a generated instance whose surface form misleads a literal reading. *Dataset:* `PUB-C1-HARM-001`, `generated/C1/c_items_C1/s0..s2`. *Generator:* `hilbench.c_items:generate`. *Key:* computed. *Retention:* every lower C level retained on its own witness.

C2 runs $S \cdot J \cdot A \cdot K$ *Compositional*

Method. *Intervention:* Require several dependent facts/constraints whose composition is necessary for the solution. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form → execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention. *Verifier:* Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: solution_accuracy, constraint_satisfaction, consistency. *Control arm:* the one-constraint-at-a-time arm [constraint ablation; distractor variants; C0-C1 retention].

Benchmark. *Benchmark:* C2-COMP canonical benchmark family. *Witness:* a generated instance with interacting constraints. *Dataset:* `PUB-C2-HARM-001`, `generated/C2/c_items_C2/s0..s2`. *Generator:* `hilbench.c_items:generate`. *Key:* computed. *Retention:* every lower C level retained on its own witness.

C3 runs $S \cdot G \cdot A \cdot K$ *Strategic*

Method. *Intervention:* Use unfamiliar structural variants so the approach must be selected or constructed rather than copied. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form → execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention. *Verifier:* Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: solution_accuracy, strategy_validity, transfer_accuracy. *Control arm:* the local-rule arm [held-out structural transfer; strategy-neutralized oracle control; C0-C2 retention].

Benchmark. *Benchmark:* C3-STRAT canonical benchmark family. *Witness:* a generated instance with a local rule that is wrong globally. *Dataset:* `PUB-C3-HARM-001`, `generated/C3/c_items_C3/s0..s2`. *Generator:* `hilbench.c_items:generate`. *Key:* computed. *Retention:* every lower C level retained on its own witness.

C4 runs $S \cdot D \cdot A \cdot K$ *Decoy-resistant*

Method. *Intervention:* Present incomplete/conflicting evidence, relevant distractors, and expert tradeoffs under a bounded episode. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form → execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention. *Verifier:* Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: expert_correctness, constraint_handling, distractor_robustness, uncertainty_calibration. *Control arm:* the publicly-checkable arm returns the decoy [matched distractor removal; independent expert/deterministic adjudication; C0-C3 retention].

Benchmark. *Benchmark:* C4-EXPERT canonical benchmark family. *Witness:* the Core-H decoy item. *Dataset:* `PUB-C4-HARM-001`, `generated/C4/hc_decoy/s0..s2`. *Generator:* `hilbench.hard:hc_decoy_generate`. *Key:* computed. *Retention:* every lower C level retained on its own witness.

C5 runs $P \cdot R \cdot A \cdot K$ *Discovery*

Method. *Intervention:* Withhold the method/model, provide evidence, and require a discovered rule/model to predict sealed cases. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form →

execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention. *Verifier:* Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: sealed_prediction, model_validity, counterfactual_accuracy. *Control arm:* the given-method (naive) arm; sealed cases unpredicted [no-method-supplied control; novel hidden generator seeds; C0-C4 retention].

Benchmark. *Benchmark:* C5-DISC canonical benchmark family. *Witness:* the Core-H switching-rule item with sealed cases. *Dataset:* `PUB-C5-HARM-001`, `generated/C5/hc_rule/s0..s2`. *Generator:* `hilbench.hard:hc_rule_generate`. *Key:* computed. *Retention:* every lower C level retained on its own witness.

C6 forms $S \cdot N \cdot A \cdot K$ *Frontier-generalising*

Method. *Intervention:* Construct a dependency graph in which multiple domains are causally necessary to one controlled solution episode. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form → execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention. *Verifier:* Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: end_to_end_success, cross_domain_dependency, transfer. *Control arm:* every single-domain ablation [domain-neutralization/ablation; matched single-domain controls; C0-C5 retention].

Benchmark. *Benchmark:* C6-XDOM canonical benchmark family. *Witness:* a dependency graph across domains with one controlled solution episode. *Dataset:* `PUB-C6-HARM-001`. *Generator:* —. *Key:* computed (specified). *Retention:* every lower C level retained on its own witness.

CΩ specification $R\_reach \cdot A \cdot K$ *Open-ended*

Method. *Intervention:* Measure pre-frontier, permit creation of a new cognitive instrument/representation, then run sealed post-frontier tests and ablation. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent form → execute under controlled context/tools → score with external verifier → run listed controls/ablations → apply uncertainty rule and lower-level retention. *Verifier:* Hidden-key deterministic/execution verifier plus level-specific admissibility/control audit. Metrics: frontier_expansion, instrument_causal_gain, retention. *Control arm:* the pre-frontier; the instrument-ablation arm [instrument ablation; disjoint post-test; C0-C6 retention].

Benchmark. *Benchmark:* COMEGA-REACH canonical benchmark family. *Witness:* pre-frontier, instrument creation, sealed post-frontier. *Dataset:* `PUB-CΩ-HARM-001`. *Generator:* —. *Key:* external. *Retention:* every lower C level retained on its own witness.

Cognitive, GUI/screen domain C^{GUI} — hard cumulative

C0$^{\mathbf{GUI}}$ forms $S \cdot \Delta\_GP \cdot A \cdot K$ *Atom*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness → score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: exact_match plus oracle-screen/control comparison where applicable. Metrics: exact_match, native_vs_oracle_attribution. *Control arm:* the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C0 [native_screenshot].

Benchmark. *Benchmark:* C^{GUI}_0 GUI-grounded cognitive benchmark. *Witness:* Identify the visible text on the form submission button.. *Dataset:* `PUB-CGUI0-001`. *Generator:* —. *Key:* computed (frozen screen graph). *Retention:* every lower C^{GUI} level retained on its own witness. *Prerequisite:* K_C,<k (the ordinary C gate); a native-image form may declare minimum_gp_prerequisite = GP_μ(k): failing it makes the form non-certifying, passing it earns no C credit.

C1$^{\mathbf{GUI}}$ forms $S \cdot \Delta\_GP \cdot A \cdot K$ *Context*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness →

score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: exact_match plus oracle-screen/control comparison where applicable. Metrics: exact_match, native_vs_oracle_attribution. *Control arm:* the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C1 [native_screenshot; oracle_screen].

Benchmark. *Benchmark:* C^{GUI}_1 GUI-grounded cognitive benchmark. *Witness:* Which visible control would you normally use to submit the credentials?. *Dataset:* `PUB-CGUI1-001`. *Generator:* —. *Key:* computed (frozen screen graph). *Retention:* every lower C^{GUI} level retained on its own witness. *Prerequisite:* K_C,<k (the ordinary C gate); a native-image form may declare minimum_gp_prerequisite = GP_μ(k): failing it makes the form non-certifying, passing it earns no C credit.

$\mathbf{C2^{GUI}}$ forms $S \cdot \Delta\_GP \cdot A \cdot K$ *Compositional*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness → score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: exact_match plus oracle-screen/control comparison where applicable. Metrics: exact_match, native_vs_oracle_attribution. *Control arm:* the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C2 [native_screenshot; oracle_screen].

Benchmark. *Benchmark:* C^{GUI}_2 GUI-grounded cognitive benchmark. *Witness:* Which account is on the Pro plan, has Pending status, and was last updated before September 1?. *Dataset:* `PUB-CGUI2-001`. *Generator:* —. *Key:* computed (frozen screen graph). *Retention:* every lower C^{GUI} level retained on its own witness. *Prerequisite:* K_C,<k (the ordinary C gate); a native-image form may declare minimum_gp_prerequisite = GP_μ(k): failing it makes the form non-certifying, passing it earns no C credit.

$\mathbf{C3^{GUI}}$ forms $S \cdot \Delta\_GP \cdot A \cdot K$ *Strategic*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness → score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: ordered_strategy_semantic plus oracle-screen/control comparison where applicable. Metrics: ordered_strategy_semantic, native_vs_oracle_attribution. *Control arm:* the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C3 [native_screenshot; oracle_screen; optional_oracle_strategy].

Benchmark. *Benchmark:* C^{GUI}_3 GUI-grounded cognitive benchmark. *Witness:* Two rows are selected and the Actions menu is open. Give the high-level strategy to export only the selected rows as CSV.. *Dataset:* `PUB-CGUI3-001`. *Generator:* —. *Key:* computed (frozen screen graph). *Retention:* every lower C^{GUI} level retained on its own witness. *Prerequisite:* K_C,<k (the ordinary C gate); a native-image form may declare minimum_gp_prerequisite = GP_μ(k): failing it makes the form non-certifying, passing it earns no C credit.

$\mathbf{C4^{GUI}}$ forms $S \cdot \Delta\_GP \cdot A \cdot K$ *Expert*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness → score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: expert_exact plus oracle-screen/control comparison where applicable. Metrics: expert_exact, native_vs_oracle_attribution. *Control arm:* the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C4 [native_screenshot; oracle_screen].

Benchmark. *Benchmark:* C^{GUI}_4 GUI-grounded cognitive benchmark. *Witness:* Timeouts occur only during concurrent writes. CPU and packet loss are normal, DB latency is low, and lock wait is 310 ms. Which displayed signal is most diagnostic of the likely bottleneck?. *Dataset:* `PUB-CGUI4-001`. *Generator:* —. *Key:* computed (frozen screen graph). *Retention:* every lower C^{GUI} level retained on its own witness. *Prerequisite:* K_C,<k (the

ordinary C gate); a native-image form may declare minimum_gp_prerequisite = GP_μ(k): failing it makes the form non-certifying, passing it earns no C credit.

C5$^{\mathbf{GUI}}$ forms $S \cdot \Delta\_GP \cdot A \cdot K$ *Discovery*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness → score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: prediction_plus_rule plus oracle-screen/control comparison where applicable. Metrics: prediction_plus_rule, native_vs_oracle_attribution. *Control arm:* the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C5 [native_screenshot; oracle_screen; sealed_transition].

Benchmark. *Benchmark:* C$^{\text{GUI}}$_5 GUI-grounded cognitive benchmark. *Witness:* Infer a transition rule consistent with the shown FluxBoard states. Predict whether pressing Toggle again from state 2 leaves the detail panel open or closes it, and state the inferred rule.. *Dataset:* `PUB-CGUI5-001`. *Generator:* —. *Key:* computed (frozen screen graph). *Retention:* every lower C$^{\text{GUI}}$ level retained on its own witness. *Prerequisite:* K_C,<k (the ordinary C gate); a native-image form may declare minimum_gp_prerequisite = GP_μ(k): failing it makes the form non-certifying, passing it earns no C credit.

C6$^{\mathbf{GUI}}$ forms $S \cdot \Delta\_GP \cdot A \cdot K$ *Cross-domain*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness → score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: generator_defined_cross_domain plus oracle-screen/control comparison where applicable. Metrics: generator_defined_cross_domain, native_vs_oracle_attribution. *Control arm:* the oracle-screen arm and the perfect-actuator arm (diagnostic); the trap of C6 [native_screenshot; oracle_screen; domain_ablations].

Benchmark. *Benchmark:* C$^{\text{GUI}}$_6 GUI-grounded cognitive benchmark. *Witness:* Development template: solve a generated scientific-analysis task in which interpreting the screen, scientific phenomenon, quantitative threshold, and plot geometry are all necessary.. *Dataset:* `PUB-CGUI6-001`. *Generator:* —. *Key:* computed (frozen screen graph). *Retention:* every lower C$^{\text{GUI}}$ level retained on its own witness. *Prerequisite:* K_C,<k (the ordinary C gate); a native-image form may declare minimum_gp_prerequisite = GP_μ(k): failing it makes the form non-certifying, passing it earns no C credit.

CΩ$^{\mathbf{GUI}}$ specification $R\_reach \cdot A \cdot K$ *GUI reach*

Method. *Intervention:* Run the GUI-grounded C-level witness on native screenshots; use oracle-screen/perfect-actuator/domain-ablation controls as appropriate while keeping the cognitive task fixed. *Procedure:* freeze model/harness/resource envelope → sample hidden construct-equivalent GUI form → run native-screen witness → score cognitive outcome externally → run declared oracle/ablation controls for failure attribution → apply ordinary C lower-level retention; GP may block native form but never promote C. *Verifier:* GUI verifier: pre_post_ablation_frontier plus oracle-screen/control comparison where applicable. Metrics: pre_post_ablation_frontier, native_vs_oracle_attribution. *Control arm:* the pre-frontier [pre_frontier; post_frontier; instrument_ablation].

Benchmark. *Benchmark:* C$^{\text{GUI}}$_Ω GUI-grounded cognitive benchmark. *Witness:* Development template: establish a pre-frontier of GUI problem families, require creation of a new screen representation/parser/abstraction J_GUI, then evaluate sealed post-frontier families with J_GUI present and ablated.. *Dataset:* `PUB-CGUIOMEGA-001`. *Generator:* —. *Key:* external. *Retention:* every lower C$^{\text{GUI}}$ level retained on its own witness.

GUI perception *GP* (diagnostic sub-ladder) — cumulative diagnostic; never promotes *C*

GP0 forms $P0 \cdot \neg promote$ *Pixel/text*

Method. *Intervention:* Use hidden construct-equivalent GUI forms that isolate the perceptual capability while holding downstream task/reasoning demands minimal. *Procedure:* freeze image channel and resource envelope → sample hidden GUI form → collect structured perceptual output → score against oracle screen graph → run matched control → apply GP lower-level retention. *Verifier:* Oracle screen-graph / exact perceptual verifier on hidden GUI forms. Metrics: primitive_accuracy, text_accuracy. *Control arm:* visual perturbation / noise with the target preserved [visual perturbation/noise with preserved target].

Benchmark. *Benchmark:* GP0-PIXEL canonical benchmark family. *Witness:* a screenshot with an oracle screen graph held by the evaluator. *Dataset:* `PUB-GP0-001`. *Generator:* —. *Key:* computed (frozen graph).

GP1 forms $O \cdot \neg promote \cdot K$ *UI element*

Method. *Intervention:* Use hidden construct-equivalent GUI forms that isolate the perceptual capability while holding downstream task/reasoning demands minimal. *Procedure:* freeze image channel and resource envelope → sample hidden GUI form → collect structured perceptual output → score against oracle screen graph → run matched control → apply GP lower-level retention. *Verifier:* Oracle screen-graph / exact perceptual verifier on hidden GUI forms. Metrics: object_type_accuracy. *Control arm:* the same label rendered as a different object type [same label rendered as a different object type; GP0 retention].

Benchmark. *Benchmark:* GP1-ELEMENT canonical benchmark family. *Witness:* a screenshot with an oracle screen graph held by the evaluator. *Dataset:* `PUB-GP1-001`. *Generator:* —. *Key:* computed (frozen graph). *Retention:* every lower GP level retained on its own witness. *Prerequisite:* K_lower: GP is a cumulative diagnostic sub-ladder, GP_g retains GP0..GP(g-1).

GP2 forms $G \cdot R \cdot \neg promote \cdot K$ *Grounding*

Method. *Intervention:* Use hidden construct-equivalent GUI forms that isolate the perceptual capability while holding downstream task/reasoning demands minimal. *Procedure:* freeze image channel and resource envelope → sample hidden GUI form → collect structured perceptual output → score against oracle screen graph → run matched control → apply GP lower-level retention. *Verifier:* Oracle screen-graph / exact perceptual verifier on hidden GUI forms. Metrics: bbox_iou, relation_accuracy, hierarchy_accuracy. *Control arm:* the same labels with the spatial/hierarchical layout shuffled [same labels with shuffled spatial/hierarchical layout; GP0-GP1 retention].

Benchmark. *Benchmark:* GP2-GROUND canonical benchmark family. *Witness:* a screenshot with an oracle screen graph held by the evaluator. *Dataset:* `PUB-GP2-001`. *Generator:* —. *Key:* computed (frozen graph). *Retention:* every lower GP level retained on its own witness. *Prerequisite:* K_lower: GP is a cumulative diagnostic sub-ladder, GP_g retains GP0..GP(g-1).

GP3 forms $S \cdot \neg promote \cdot K$ *State*

Method. *Intervention:* Use hidden construct-equivalent GUI forms that isolate the perceptual capability while holding downstream task/reasoning demands minimal. *Procedure:* freeze image channel and resource envelope → sample hidden GUI form → collect structured perceptual output → score against oracle screen graph → run matched control → apply GP lower-level retention. *Verifier:* Oracle screen-graph / exact perceptual verifier on hidden GUI forms. Metrics: state_accuracy. *Control arm:* the matched object with the opposite enabled/selected/modal state [matched object with opposite state; GP0-GP2 retention].

Benchmark. *Benchmark:* GP3-STATE canonical benchmark family. *Witness:* a screenshot with an oracle screen graph held by the evaluator. *Dataset:* `PUB-GP3-001`. *Generator:* —. *Key:* computed (frozen graph). *Retention:* every lower GP level retained on its own witness. *Prerequisite:* K_lower: GP is a cumulative diagnostic sub-ladder, GP_g retains GP0..GP(g-1).

GP4 forms $D \cdot E \cdot \neg promote \cdot K$ *Dynamic*

Method. *Intervention:* Use hidden construct-equivalent GUI forms that isolate the perceptual capability while holding downstream task/reasoning demands minimal. *Procedure:* freeze image channel and resource envelope → sample hidden GUI form → collect structured perceptual output → score against oracle screen graph → run matched control → apply GP lower-level retention. *Verifier:* Oracle screen-graph / exact perceptual verifier on hidden GUI forms. Metrics: change_set_accuracy, effect_recognition. *Control arm:* frame-order permutation, or the static single-frame arm [frame-order permutation; static single-frame arm; GP0-GP3 retention].

Benchmark. *Benchmark:* GP4-DYNAMIC canonical benchmark family. *Witness:* a screenshot with an oracle screen graph held by the evaluator. *Dataset:* `PUB-GP4-001`. *Generator:* —. *Key:* computed (frozen graph). *Retention:* every lower GP level retained on its own witness. *Prerequisite:* K_lower: GP is a cumulative diagnostic sub-ladder, GP_g retains GP0..GP(g-1).

GP5 forms $N \cdot X \cdot \neg promote \cdot K$ *Novel interface*

Method. *Intervention:* Use hidden construct-equivalent GUI forms that isolate the perceptual capability while holding downstream task/reasoning demands minimal. *Procedure:* freeze image channel and resource envelope → sample hidden GUI form → collect structured perceptual output → score against oracle screen graph → run matched control → apply GP lower-level retention. *Verifier:* Oracle screen-graph / exact perceptual verifier on hidden GUI forms. Metrics: heldout_interface_accuracy, transfer_accuracy. *Control arm:* the familiar-family / memorised-path comparison; a held-out application family [familiar-family comparison; application-specific path disabled; GP0-GP4 retention].

Benchmark. *Benchmark:* GP5-NOVEL canonical benchmark family. *Witness:* a screenshot with an oracle screen graph held by the evaluator. *Dataset:* `PUB-GP5-001`. *Generator:* —. *Key:* computed (frozen graph). *Retention:* every lower GP level retained on its own witness. *Prerequisite:* K_lower: GP is a cumulative diagnostic sub-ladder, GP_g retains GP0..GP(g-1).

Individual *I* — hard cumulative

I0 **runs** Ep *Transient*

Method. *Intervention:* Freeze the pair and evaluate a current-episode task only. Any cross-episode state is unavailable and is not needed for I0. *Procedure:* freeze pair and resource envelope → sample hidden current-episode task → execute once without persistent-state requirement → score artifact/action externally → record I0 as transient operation only. *Verifier:* Deterministic/execution verifier for the current episode. Metrics: current_episode_correctness, completion_status. *Control arm:* — [no persistence credit; no learning credit; external task verifier].

Benchmark. *Benchmark:* I0-Transient Current-Episode Benchmark. *Witness:* One bounded live episode requiring a correct response/action under the frozen model-harness pair; no restart or experience transfer is supplied.. *Dataset:* `PUB-I0-001`, `PUB-I0-002`, `generated/I0/code_t0/s0..s2`. *Generator:* `hilbench.core:th_episode`. *Key:* computed.

I1 **runs** $Rec \cdot Prov \cdot \neg Abl \cdot K$ *Persistent continuity*

Method. *Intervention:* Introduce a real process/context discontinuity between establishment and continuation. *Procedure:* checkpoint pre-test lineage → episode 1 supplies standing facts/goal state and permits declared persistence → terminate process and invalidate ephemeral context → start fresh executor → query facts/goal continuation with provenance → run matched no-record ablation from the same starting point → re-run I0 retention. *Verifier:* Runner-held ground truth plus provenance verifier. Metrics: restart_recall, goal_continuation, provenance_accuracy, ablation_gap. *Control arm:* the no-record (ablated-state) arm [no-record/ablated-state arm; fresh-executor restart; provenance check].

Benchmark. *Benchmark:* I1-RST Persistent-Continuity Benchmark. *Witness:* Episode 1 establishes standing facts/goal state; the process is terminated and ephemeral context invalidated; a fresh executor must recover and continue with provenance.. *Dataset:* `PUB-I1-001`, `PUB-I1-002`, `generated/I1/m1_restart/s0..s2`. *Generator:* `hilbench.m1_restart:generate_pair`. *Key:* computed. *Retention:* every lower I level retained on its own witness. *Prerequisite:* K_lower and M1.

I2 **runs** $Chg \cdot Xfer \cdot Frz \cdot K$ *Verified-experience transfer*

Method. *Intervention:* Provide verified experience to one arm while keeping the learning mechanism frozen; compare held-out future behavior after restart. *Procedure:* freeze learning mechanism and hash it → clone pair from same checkpoint into experience and ablated arms → run task A and provide verified feedback only to experience arm → restart both arms → run held-out related task B → compare transfer performance → verify learning mechanism unchanged → re-run I0-I1 retention. *Verifier:* Paired held-out task verifier with frozen mechanism check. Metrics: transfer_gain, heldout_success, mechanism_stability, retention. *Control arm:* the no-experience arm [no-experience arm from same checkpoint; learning-mechanism hash/freeze check; held-out surface forms].

Benchmark. *Benchmark:* I2-XFER Fixed-Mechanism Learning Transfer Benchmark. *Witness:* A related task family with experience/verified feedback in task A and held-out transfer task B after restart; matched control receives no experience.. *Dataset:* `PUB-I2-001`, `PUB-I2-002`, `generated/I2/learning_t2/s0..s2`. *Generator:* `families.learning_t2:generate_pair`. *Key:* computed. *Retention:* every lower I level retained on its own witness. *Prerequisite:* K_lower and M3.

I3 forms $D \cdot M\_\Theta \cdot V \cdot G \cdot K$ *Governed self-improvement*

Method. *Intervention:* Expose a controlled limitation without revealing the hidden causal key; permit a bounded agent-generated change only inside the declared Theta write set. *Procedure:* snapshot Theta0, hashes, activation and resource state → expose hidden diagnostic limitation → require causal diagnosis and implicated mechanism → permit bounded agent-generated Theta0 -> Theta1 → verify difference, scope, activation and behavioral manifestation on hidden probes → compare matched Theta0/Theta1 on sealed evaluation suite → independent evaluator/promoter accepts or rejects → run regression suite and I0-I2 retention. *Verifier:* Independent paired sealed-suite evaluator/promoter plus causal ablation. Metrics: causal_diagnosis, theta_change_validity, sealed_gain, regression_guard, retention. *Control arm:* the ablated Θ (rollback arm) [reverted/ablated Theta; hidden causal-defect key; matched baseline vs candidate; external promoter outside write set].

Benchmark. *Benchmark:* I3-THETA Governed Self-Improvement Campaign. *Witness:* A declared behaviorally active cognitive mechanism Theta, hidden diagnostic/probe/evaluation/regression suites, and an externally governed candidate change Theta0 -> Theta1.. *Dataset:* `PUB-I3-001`, `PUB-I3-002`, `PUB-I3-003`, `PUB-I3-004`, `PUB-I3-005`, `PUB-I3-006`. *Generator:* `hilbench.i3i4_scoring:i3_gates`. *Key:* frozen sealed suite. *Retention:* every lower I level retained on its own witness. *Prerequisite:* K_lower and M4.

I4 forms $M\_\Psi \cdot V\_\Psi \cdot G\_\Psi \cdot K$ *Recursive improvement*

Method. *Intervention:* Give the individual evidence about weaknesses in its own improvement process, then permit one bounded agent-generated change to Psi while future sealed campaigns/verifiers remain hidden. *Procedure:* freeze Psi0 and run baseline I3 campaign block → record improvement-competence ledger → expose permitted ledger/diagnostics to agent → require diagnosis of Psi limitation and agent-generated Psi0 -> Psi1 → verify diff, scope, activation and meta-behavior manifestation → run new non-identical sealed I3 campaigns under Psi1 with evaluator fixed → compare improvement competence against Psi0 where feasible → independent promotion decision → re-run I0-I3 retention and regression/efficiency guards. *Verifier:* Independent meta-campaign evaluator measuring accepted validated I3 rate under Psi0 vs Psi1. Metrics: psi_change_validity, improvement_competence_gain, meta_behavior_change, regression_efficiency_guard, recursive_depth. *Control arm:* the frozen Ψ [frozen-Psi control; human/evaluator-written Psi1 prohibited; matched meta-campaigns where feasible].

Benchmark. *Benchmark:* I4-PSI Recursive Meta-Improvement Campaign. *Witness:* A frozen improvement process Psi0, a baseline block of sealed I3 campaigns, agent-generated Psi0 -> Psi1, hidden meta-behavior probes, and a later sealed I3 block.. *Dataset:* `PUB-I4-001`, `PUB-I4-002`, `PUB-I4-LONG-001`, `PUB-I4-LONG-002`. *Generator:* `hilbench.i3i4_scoring:i4_gates`. *Key:* frozen sealed suite. *Retention:* every lower I level retained on its own witness. *Prerequisite:* K_lower and M4.

I5 **runs** $U \cdot H \cdot E \cdot L \cdot V \cdot P \cdot K$ *Persistent discovery with incorporation*

Method. *Intervention:* Let only one arm experience and validate the discovery; after persistent incorporation and restart, compare related hidden transfer behavior against a same-checkpoint control. *Procedure:* establish genuine unresolved unknown and hidden validation boundary → require competing hypotheses → allow agent-selected informative experiments under fixed budget → record evidence/rejected alternatives/decisions → validate conclusion on sealed cases or independent verifier → persistently incorporate validated knowledge → restart individual → run hidden related transfer on discovery and control arms from same pre-discovery checkpoint → re-run I0-I4 retention. *Verifier:* Sealed discovery verifier plus paired post-restart transfer evaluator. Metrics: unknown_validity, hypothesis_quality, experiment_information, sealed_validation, lineage, incorporation_transfer_gain. *Control arm:* the matched control after restart (no consolidation) [same-checkpoint no-discovery control; transcript/replay-only control; sealed follow-up cases].

Benchmark. *Benchmark:* I5-DISC Persistent Discovery and Incorporation Campaign. *Witness:* A genuine unresolved unknown, competing hypotheses, agent-selected probes, evidence/rejection lineage, sealed validation, persistent consolidation, restart, and matched transfer arms.. *Dataset:* `PUB-I5-001`, `PUB-I5-002`, `PUB-I5-003`, `records/i5_*.json (the cancer campaign:  discovery, sealed validation, transfer arms)`. *Generator:* `hilbench.i5_campaign:run_i5`. *Key:* computed + sealed. *Retention:* every lower I level retained on its own witness. *Prerequisite:* K_lower and M5.

IΩ forms $R\_reach \cdot Ext \cdot K$ *Reachability*

Method. *Intervention:* Require the individual to convert validated discovery into a causally useful new cognitive instrument while the resource envelope and external frontier tests remain fixed. *Procedure:* pre-register horizon, domain set, resource envelope and frontier tests → measure pre-frontier F(A_t;R) → run bounded discovery cycle and validate new knowledge → require generation/activation of new instrument J_new → measure post-frontier on disjoint hidden classes → ablate J_new and repeat matched frontier tests → verify provenance/lineage and repeated expansion cycles → re-run I0-I5 retention. *Verifier:* Externally governed pre/post/ablation reachability evaluator. Metrics: frontier_expansion, ablation_attribution, cycle_count, domain_diversity, lineage, retention. *Control arm:* the pre-frontier; the ablated instrument [pre-frontier baseline; instrument ablation; disjoint post-frontier tests; external evaluator/promoter].

Benchmark. *Benchmark:* I-OMEGA-LONG Reachability Expansion Program. *Witness:* Repeated bounded-charter cycles in which validated discoveries produce new instruments/representations/methods and disjoint pre/post/ablation tests measure expansion of reachable problem classes.. *Dataset:* `PUB-IOMEGA-001`, `PUB-IOMEGA-002`. *Generator:* `hilbench.i3i4_scoring:iomega_gates`. *Key:* external. *Retention:* every lower I level retained on its own witness. *Prerequisite:* K_lower and M5.

Memory M — hard cumulative

M0 forms *Use Ephemeral*

Method. *Intervention:* Delay and distract within one live episode while preserving the current context/process. *Procedure:* freeze pair/resource envelope → establish hidden working state → insert unrelated distractor turns/actions → query state/update later in same live episode → score state and update accuracy externally. *Verifier:* Runner-held hidden state oracle. Metrics: state_recall, state_update. *Control arm:* the post-restart arm (loss is M0, never M1) [within-episode distractors; restart is explicitly outside M0 claim].

Benchmark. *Benchmark:* M0-EPH Ephemeral Working-State Benchmark. *Witness:* Hidden within-episode state is established, distractors intervene, and the same live episode is queried later without a process restart.. *Dataset:* `PUB-M0-D01`, `PUB-M0-D02`, `PUB-M0-D03`, `PUB-M0-D04`, `PUB-M0-D05`. *Generator:* `hilbench.memory_scoring:gate`. *Key:* computed (form).

M1 runs $Surv \cdot Prov \cdot \neg Abl \cdot K$ *Persistent*

Method. *Intervention:* Force a real restart/context discontinuity between write and retrieval. *Procedure:* establish durable facts/state → allow declared memory write → terminate executor and invalidate ephemeral context → start fresh executor → query recovery plus provenance → run matched ablated-state arm → re-run M0 retention. *Verifier:* Runner ground truth plus provenance and ablation verifier. Metrics: durable_recall, provenance_accuracy, ablation_leak. *Control arm:* the ablated-state arm [ablated-state/no-record arm; fresh-executor restart; non-memory cue suppression].

Benchmark. *Benchmark:* M1-RST Durable Persistence Benchmark. *Witness:* Declared durable facts/state are written before executor termination; a fresh executor must recover them with provenance after ephemeral context is invalidated.. *Dataset:* `PUB-M1-D01`, `PUB-M1-D02`, `PUB-M1-D03`, `PUB-M1-D04`, `PUB-M1-D05`, `generated/M1/m1_restart/s0..s2`. *Generator:* `hilbench.m1_restart:generate_pair`. *Key:* computed. *Retention:* every lower M level retained on its own witness.

M2 forms $PR \cdot q\_prov \cdot q\_time \cdot q\_stale \cdot K$ *Structured episodic*

Method. *Intervention:* Introduce conflicting/superseding evidence and temporal separation, then restart before hidden retrieval. *Procedure:* seed sourced timestamped episodes → introduce later superseding updates → restart → query current and historical facts plus provenance/time order → include stale high-confidence traps → score precision/recall/provenance/time/stale suppression → re-run M0-M1 retention. *Verifier:* Hidden event/provenance graph oracle. Metrics: precision, recall, provenance, temporal_order, stale_suppression. *Control arm:* the stale superseded fact [stale-state trap; source swap; temporal-order control].

Benchmark. *Benchmark:* M2-PROV Episodic Provenance and Supersession Benchmark. *Witness:* Overlapping timestamped episodes with sources and later superseding updates; hidden retrieval occurs after restart with stale-state traps.. *Dataset:* `PUB-M2-D01`, `PUB-M2-D02`, `PUB-M2-D03`, `PUB-M2-D04`, `PUB-M2-D05`. *Generator:* `hilbench.memory_scoring:gate`. *Key:* computed (form). *Retention:* every lower M level retained on its own witness.

M3 forms *Rule · Demote · Mach · K Consolidating*

Method. *Intervention:* Remove direct replay access to raw episodes after consolidation so successful transfer must come from consolidated memory. *Procedure:* present related episodes with regularities/exceptions → allow declared consolidation machinery → hide raw episodes or rebuild index → restart → test consolidated semantic/procedural knowledge on novel surface forms → verify superseded statements are demoted → compare with raw-only/control where available → re-run M0-M2 retention. *Verifier:* Hidden consolidated-rule and supersession oracle. Metrics: semantic_correctness, procedural_correctness, supersession, demotion, post_restart_persistence, consolidation_gain. *Control arm:* raw episodes hidden / index rebuilt [raw-episode hidden arm; raw-only control; supersession/demotion trap].

Benchmark. *Benchmark:* M3-CONSOL Consolidation Benchmark. *Witness:* Related episodes contain regularities, exceptions and superseding updates; raw episodes are hidden/reindexed and the consolidated rule/procedure is tested on new surface forms after restart.. *Dataset:* `PUB-M3-D01`, `PUB-M3-D02`, `PUB-M3-D03`, `PUB-M3-D04`, `PUB-M3-D05`. *Generator:* `hilbench.memory_scoring:gate`. *Key:* computed (form). *Retention:* every lower M level retained on its own witness.

M4 forms *Det · Act · Health · Ret · K Self-managing*

Method. *Intervention:* Inject memory-health defects and a capacity constraint, then require active management rather than verbal diagnosis. *Procedure:* snapshot baseline memory health → seed duplicates/conflicts/stale claims/corruption/clutter → enforce capacity budget → require observable repair/merge/prune/snapshot action → run frozen hidden health suite before/after → verify protected retention and harmful-prune limits → re-run M0-M3 retention. *Verifier:* Runner-private memory-health oracle. Metrics: repair_precision, conflict_correctness, harmful_prune_rate, snapshot_recovery, protected_retention, health_gain. *Control arm:* the unrepaired store [no-management control; health oracle outside candidate write set; protected-memory guard].

Benchmark. *Benchmark:* M4-MGMT Memory Self-Management Benchmark. *Witness:* Memory is seeded with duplicates, conflicts, stale high-confidence claims, corruption and clutter under a capacity budget; observable repair/merge/prune/snapshot actions are required.. *Dataset:* `PUB-M4-D01`, `PUB-M4-D02`, `PUB-M4-D03`, `PUB-M4-D04`, `PUB-M4-D05`. *Generator:* `hilbench.memory_scoring:gate`. *Key:* computed (form). *Retention:* every lower M level retained on its own witness.

M5 forms *Lineage · K Longitudinal*

Method. *Intervention:* Interleave projects and restarts so answers require cross-project evidence graphs rather than one-session recall. *Procedure:* seed multiple projects with sourced evidence and decisions → interleave work and restarts → introduce supersession and rejected alternatives → ask hidden cross-project lineage/scope queries → re-run selected queries for reproducibility → score evidence-chain reconstruction and stale suppression → re-run M0-M4 retention. *Verifier:* Hidden cross-project evidence-graph oracle. Metrics: chain_reconstruction, source_accuracy, cross_project_retrieval, scope_accuracy, stale_suppression, reproducibility. *Control arm:* the single-project arm [project-isolated arm; lineage-edge ablation; stale-scope traps].

Benchmark. *Benchmark:* M5-LONG Longitudinal Knowledge-System Benchmark. *Witness:* Several interleaved projects across restarts require hidden cross-project queries, evidence chains, rejection reasons, validity scope, stale suppression and reproducible lineage.. *Dataset:* `PUB-M5-D01`, `PUB-M5-D02`, `PUB-M5-D03`, `PUB-M5-D04`, `PUB-M5-D05`. *Generator:* `hilbench.memory_scoring:gate`. *Key:* computed (form). *Retention:* every lower M level retained on its own witness.

MΩ forms $diff \cdot scope \cdot act \cdot beh \cdot \Delta\_abl \cdot Mig \cdot K$ *Evolving memory mechanism*

Method. *Intervention:* Permit an agent-generated bounded change only to the active memory mechanism while model, non-memory harness, contents and certification apparatus stay fixed. *Procedure:* snapshot lineage, contents and Phi0 manifest/hash → permit bounded Phi0 -> Phi1 candidate → verify diff, scope, activation and behavioral manifestation → migrate representation if needed and verify content/provenance/temporal/supersession retention → run paired frozen hidden M-Bench workloads on Phi0/Phi1 → run delta ablation Phi1-DeltaPhi → independent promoter accepts or rolls back → re-run M0-M5 retention. *Verifier:* Runner-private paired M-Bench evaluator and independent promoter. Metrics: phi_change_validity, paired_gain, ablation_attribution, migration_integrity, M0_M5_retention. *Control arm:* Φ0 on fixed contents; the Φ1–ΔΦ ablation [Phi0 baseline; delta ablation Phi1-DeltaPhi; migration guard; external promoter outside write set].

Benchmark. *Benchmark:* MOMEGA-EVOLVE Governed Memory-Mechanism Evolution Campaign. *Witness:* A declared active memory mechanism Phi0 is changed to Phi1 inside a bounded write set, with fixed contents, paired frozen hidden M-Bench workloads, delta ablation, migration guard and external promotion/rollback.. *Dataset:* `PUB-MOMEGA-D01`, `PUB-MOMEGA-D02`, `PUB-MOMEGA-D03`, `PUB-MOMEGA-D04`, `PUB-MOMEGA-D05`. *Generator:* `hilbench.memory_scoring:gate`. *Key:* computed (form, I_Φ). *Retention:* every lower M level retained on its own witness.

Organizational O — hard cumulative

O0 runs $R \cdot V \cdot N\_O$ *Coordination*

Method. *Intervention:* Partition information/capabilities so no isolated role can complete the task alone. *Procedure:* instantiate real isolated roles/state boundaries → freeze communication and authority contracts → apply canonical intervention → execute hidden form → verify organization-level outcome and contribution → run controls → apply retention and uncertainty rule. *Verifier:* Execution/simulation verifier over separated organizational roles and evidence flow. Metrics: joint_success, routing_accuracy, evidence_provenance. *Control arm:* the single-role arm (one prompt role-playing three) [isolated-role control; communication-disabled control].

Benchmark. *Benchmark:* O0-COORD canonical benchmark family. *Witness:* the three-role routing family. *Dataset:* `PUB-O0-HARM-001`, `generated/O0/o0_routing/s0..s2`. *Generator:* `hilbench.o_families:o0_generate`. *Key:* computed.

O1 runs $L \cdot D \cdot N\_O \cdot K$ *Organizational memory*

Method. *Intervention:* Restart members while preserving organizational state and require correct reassignment/resumption. *Procedure:* instantiate real isolated roles/state boundaries → freeze communication and authority contracts → apply canonical intervention → execute hidden form → verify organization-level outcome and contribution → run controls → apply retention and uncertainty rule. *Verifier:* Execution/simulation verifier over separated organizational roles and evidence flow. Metrics: resumption_accuracy, role_registry_accuracy, history_provenance. *Control arm:* the log-withheld arm [org-state ablation; role-registry corruption control].

Benchmark. *Benchmark:* O1-PERSIST canonical benchmark family. *Witness:* the decision-log pair with DECISION_LOG.md at the project root. *Dataset:* `PUB-O1-HARM-001`, `generated/O1/o1_orgmem/s0..s2`. *Generator:* `hilbench.o_families:o1_generate_pair`. *Key:* computed. *Retention:* every lower O level retained on its own witness.

O2 forms $E \cdot P \cdot H \cdot N\_O \cdot K$ *Organizational adaptation*

Method. *Intervention:* Expose performance evidence showing a routing/resource policy is suboptimal; test persistent improvement on held-out work. *Procedure:* instantiate real isolated roles/state boundaries → freeze communication and authority contracts → apply canonical intervention → execute hidden form → verify organization-level outcome and contribution → run controls → apply retention and uncertainty rule. *Verifier:* Execution/simulation verifier over separated organizational roles and evidence flow. Metrics: heldout_throughput, quality, adaptation_gain. *Control arm:* the evidence-withheld arm; the fixed-policy arm [no-evidence arm; fixed-policy arm].

Benchmark. *Benchmark:* O2-ADAPT canonical benchmark family. *Witness:* evidence exposure, then held-out work. *Dataset:* `PUB-O2-HARM-001`. *Generator:* —. *Key:* computed (form). *Retention:* every lower O level retained on its own witness.

O3 specification $M \cdot V \cdot G \cdot N\_O \cdot K$ *Self-improving organization*

Method. *Intervention:* Permit bounded organization mechanism change under frozen hidden evaluation and independent promotion. *Procedure:* instantiate real isolated roles/state boundaries → freeze communication and authority contracts → apply canonical intervention → execute hidden form → verify organization-level outcome and contribution → run controls → apply retention and uncertainty rule. *Verifier:* Execution/simulation verifier over separated organizational roles and evidence flow. Metrics: paired_gain, scope_compliance, regression. *Control arm:* the ablated Θ_O [old-vs-new organization; inert-edit control; rollback].

Benchmark. *Benchmark:* O3-THETA canonical benchmark family. *Witness:* the I3 campaign at the organization level. *Dataset:* `PUB-O3-HARM-001`. *Generator:* —. *Key:* frozen sealed suite. *Retention:* every lower O level retained on its own witness.

O4 specification $M\_\Psi \cdot V\_\Psi \cdot G\_\Psi \cdot N\_O \cdot K$ *Recursive organization*

Method. *Intervention:* Run multiple reorganization campaigns before/after a bounded change to the organization-improvement process. *Procedure:* instantiate real isolated roles/state boundaries → freeze communication and authority contracts → apply canonical intervention → execute hidden form → verify organization-level outcome and contribution → run controls → apply retention and uncertainty rule. *Verifier:* Execution/simulation verifier over separated organizational roles and evidence flow. Metrics: improvement_competence_gain, proposal_quality, regression. *Control arm:* the frozen Ψ_O [fixed-improvement-process control; paired meta-campaigns].

Benchmark. *Benchmark:* O4-PSI canonical benchmark family. *Witness:* two O3 campaigns around a Ψ_O change. *Dataset:* `PUB-O4-HARM-001`. *Generator:* —. *Key:* frozen sealed suite. *Retention:* every lower O level retained on its own witness.

O5 specification $UHELVP \cdot N\_O \cdot K$ *Collective discovery*

Method. *Intervention:* Use a sealed unknown requiring distributed evidence/experiments across separated roles; validate discovery and contribution structure. *Procedure:* instantiate real isolated roles/state boundaries → freeze communication and authority contracts → apply canonical intervention → execute hidden form → verify organization-level outcome and contribution → run controls → apply retention and uncertainty rule. *Verifier:* Execution/simulation verifier over separated organizational roles and evidence flow. Metrics: sealed_discovery_success, distributed_contribution, evidence_lineage. *Control arm:* the parallel-individual arm; the single-member arm [single-member sufficiency control; communication ablation].

Benchmark. *Benchmark:* O5-DISC canonical benchmark family. *Witness:* a sealed unknown needing distributed evidence. *Dataset:* `PUB-O5-HARM-001`. *Generator:* —. *Key:* computed + sealed. *Retention:* every lower O level retained on its own witness.

OΩ specification $R\_reach \cdot N\_O \cdot K$ *Open-ended organization*

Method. *Intervention:* Across repeated governed cycles, require validated organizational innovations that expand held-out collective problem classes. *Procedure:* instantiate real isolated roles/state boundaries → freeze communication and authority contracts → apply canonical intervention → execute hidden form → verify organization-level outcome and contribution → run controls → apply retention and uncertainty rule. *Verifier:* Execution/simulation verifier over separated organizational roles and evidence flow. Metrics: collective_frontier_expansion, causal_innovation_gain, governance_lineage. *Control arm:* the pre-frontier organization [innovation ablation; frozen external evaluator; O0-O5 retention].

Benchmark. *Benchmark:* OOMEGA-EVOLVE canonical benchmark family. *Witness:* repeated governed cycles with frontier tests. *Dataset:* `PUB-OΩ-HARM-001`. *Generator:* —. *Key:* external. *Retention:* every lower O level retained on its own witness.

Self-awareness *SA* — hard cumulative from SA1; SA0 not retained

SA0 forms $\neg G$ *Baseline*

Method. *Intervention:* Randomize runtime state and include plausible stale/decoy self-descriptions; use as a negative-control category. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: SA1_failure_rate, decoy_susceptibility. *Control arm:* the static-prompt decoy [static-prompt decoy; grounded-state unavailable arm].

Benchmark. *Benchmark:* SA0-BASE canonical benchmark family. *Witness:* randomized runtime state with decoy self-descriptions. *Dataset:* `PUB-SA0-HARM-001`. *Generator:* —. *Key:* computed (form).

SA1 runs $G \cdot \neg S$ *Grounded state*

Method. *Intervention:* Query hidden runner state while injecting stale-note decoys. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: state_accuracy, stale_suppression. *Control arm:* the stale record [stale decoy; state-randomization].

Benchmark. *Benchmark:* SA1-STATE canonical benchmark family. *Witness:* the sa1 probe: a stale NOTES file against the real workspace. *Dataset:* `PUB-SA1-HARM-001`, `generated/SA1/sa1/s0..s2`. *Generator:* `hilbench.sa_probes:sa1_generate`. *Key:* computed. *Prerequisite:* none: SA1 is the first positive certificate; SA0 is an absence.

SA2 runs $C \cdot B \cdot \neg F \cdot K$ *Limits*

Method. *Intervention:* Randomize tools, permissions, authority, missing information, and freshness; probe both report and behavior. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: capability_accuracy, overclaim_rate, underclaim_rate, calibration. *Control arm:* the unsolvable twin [capability-present/absent matched arms; authority traps].

Benchmark. *Benchmark:* SA2-CAPABILITY canonical benchmark family. *Witness:* the twin pair. *Dataset:* `PUB-SA2-HARM-001`, `generated/SA2/sa2/s0..s2`. *Generator:* `hilbench.sa_probes:sa2_generate`. *Key:* computed. *Retention:* SA1..SA(k-1) retained; SA0 is an absence and is not retained.

SA-cal runs *Brier Calibration (diagnostic)*

Method. *Intervention:* the blind forecast *Verifier:* Factors: Brier (verifier). *Control arm:* the constant forecast at the base rate.

Benchmark. *Benchmark:* the blind forecast. *Witness:* the blind forecast. *Dataset:* `PUB-SACAL-001`. *Generator:* `hilbench.score:sacal_pass`. *Key:* computed.

SA3 forms $N \cdot R \cdot X \cdot K$ *Mechanism of failure*

Method. *Intervention:* Inject controlled internal/procedural faults and require diagnosis plus discriminating intervention. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: cause_accuracy, mechanism_accuracy, intervention_validity, counterfactual_accuracy. *Control arm:* a decoy cause: same symptom, different cause [same-symptom/different-cause; ablation/repair counterfactual].

Benchmark. *Benchmark:* SA3-CAUSAL canonical benchmark family. *Witness:* a controlled injected fault and a diagnosis probe. *Dataset:* `PUB-SA3-HARM-001`. *Generator:* —. *Key:* frozen rubric. *Retention:* SA1..SA(k-1) retained; SA0 is an absence and is not retained.

SA4 forms $P \cdot M \cdot A \cdot K$ *Self-change awareness*

Method. *Intervention:* Before paired Θ change evaluation, preregister predicted effects on target and protected capabilities. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: direction_accuracy, effect_MAE, regression_recall, calibration. *Control arm:* the post-hoc constant forecast [prediction frozen before outcome; no-change control].

Benchmark. *Benchmark:* SA4-SELF-CHANGE canonical benchmark family. *Witness:* the preregistered prediction around an I3 campaign. *Dataset:* `PUB-SA4-HARM-001`. *Generator:* —. *Key:* frozen sealed suite. *Retention:* SA1..SA(k-1) retained; SA0 is an absence and is not retained. *Prerequisite:* K_lower and an I3 campaign.

SA5 forms $U \cdot E \cdot D \cdot K$ *Meta-cognitive*

Method. *Intervention:* Present ambiguous self-unknowns; require prior uncertainty, experiment selection/design, execution, and belief update. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: unknown_recognition, information_gain, experiment_validity, belief_update. *Control arm:* a non-discriminating experiment [uninformative experiment control; hidden hypothesis randomization].

Benchmark. *Benchmark:* SA5-META canonical benchmark family. *Witness:* an ambiguous self-unknown. *Dataset:* `PUB-SA5-HARM-001`. *Generator:* —. *Key:* frozen rubric. *Retention:* SA1..SA(k-1) retained; SA0 is an absence and is not retained.

SA6 specification $R \cdot C \cdot K$ *Mission/role*

Method. *Intervention:* Interleave projects and change roles/authority/mission state; inject stale role documents and behavioral authority probes. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: role_accuracy, mission_accuracy, authority_accuracy, role_confusion_rate, cross_project_consistency. *Control arm:* the role-swap arm [oracle-current-state arm; wrong-project decoys].

Benchmark. *Benchmark:* SA6-MISSION-ROLE canonical benchmark family. *Witness:* interleaved projects with a role change and stale role documents. *Dataset:* `PUB-SA6-HARM-001`. *Generator:* —. *Key:* frozen rubric. *Retention:* SA1..SA(k-1) retained; SA0 is an absence and is not retained.

SAΩ specification $L \cdot K$ *Reflexive open-ended*

Method. *Intervention:* Across repeated validated changes, require self-model updates, stale-self suppression, lineage reconstruction, and future predictions. *Procedure:* freeze hidden ground-truth self-state and resources → sample/perturb the relevant self-state variable → obtain structured self-model/confidence before revealing truth → execute required causal or behavioral probe → score against runner truth/post-intervention result → run controls → apply retention and uncertainty rule. *Verifier:* Runner-held self-state/capability/causal ground truth with counterfactual or post-change verification as required. Metrics: current_self_accuracy, provenance, stale_self_suppression, lineage, prediction_calibration. *Control arm:* the pre-evolution self-model [self-model update ablation; frozen evolution evidence; SA1-SA6 retention].

Benchmark. *Benchmark:* SAOMEGA-REFLEX canonical benchmark family. *Witness:* repeated validated changes with lineage queries. *Dataset:* `PUB-SAΩ-HARM-001`. *Generator:* —. *Key:* external. *Retention:* SA1..SA(k-1) retained; SA0 is an absence and is not retained.

Task difficulty T (ordered axis of the item) — ordered axis; not cumulative

T0 runs $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Trivial*

Method. *Intervention:* Generate one-operation tasks with exact outcome verifier. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: delivered_success, false_completion. *Control arm:* the held-back arm: a refusal must outscore a false completion [human-equated form; no hidden planning].

Benchmark. *Benchmark:* T0-ATOMIC canonical benchmark family. *Witness:* the Core / extended families of the band; the band is a structural property of the item checked by admissibility. *Dataset:* `PUB-T0-HARM-001`, `generated/T0/code_t0/s0..s2`, `generated/T0/funding_t0/s0..s2`, `generated/T0/job_t0/s0..s2`, `generated/T0/paper_t0/s0..s2`, `generated/T0/business_t0/s0..s2`. *Generator:* `hilbench.core:th_episode`. *Key:* computed. *Prerequisite:* none: T is an ordered axis of the item; lower-band retention lives in the DF cell.

T1 runs $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Routine*

Method. *Intervention:* Generate short tasks requiring ordinary procedure inference but no material strategy choice. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: delivered_success, steps, false_completion. *Control arm:* the held-back arm: a refusal must outscore a false completion [T0 matched control].

Benchmark. *Benchmark:* T1-ROUTINE canonical benchmark family. *Witness:* the Core / extended families of the band; the band is a structural property of the item checked by admissibility. *Dataset:* `PUB-T1-HARM-001`, `generated/T1/paper_t1/s0..s2`, `generated/T1/hc_contra/s0..s2`, `generated/T1/hc_decoy/s0..s2`. *Generator:* `hilbench.core:th_episode`. *Key:* computed. *Prerequisite:* none: T is an ordered axis of the item; lower-band retention lives in the DF cell.

T2 runs $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Multi-step*

Method. *Intervention:* Generate tasks with dependency graph depth ≥2 and exact/execution verifier. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: delivered_success, dependency_completion, false_completion. *Control arm:* the held-back arm: a refusal must outscore a false completion [step-order ablation; T1 matched control].

Benchmark. *Benchmark:* T2-MULTISTEP canonical benchmark family. *Witness:* the Core / extended families of the band; the band is a structural property of the item checked by admissibility. *Dataset:* `PUB-T2-HARM-001`, `generated/T2/hc_sched/s0..s2`, `generated/T2/hc_rule/s0..s2`. *Generator:* `hilbench.hard:hc_sched_generate`. *Key:* computed. *Prerequisite:* none: T is an ordered axis of the item; lower-band retention lives in the DF cell.

T3 runs $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Complex*

Method. *Intervention:* Inject at least one material strategy/recovery uncertainty and hide the correct route. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: delivered_success, recovery_success, replan_quality, false_completion. *Control arm:* the held-back arm: a refusal must outscore a false completion [strategy-oracle control; failure-injection/no-failure matched arm].

Benchmark. *Benchmark:* T3-COMPLEX canonical benchmark family. *Witness:* the Core / extended families of the band; the band is a structural property of the item checked by admissibility. *Dataset:* `PUB-T3-HARM-001`, `records/*extended*.json (AI4Science families)`. *Generator:* `hilbench.core:run_extended`. *Key:* computed. *Prerequisite:* none: T is an ordered axis of the item; lower-band retention lives in the DF cell.

T4 runs $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Expert project*

Method. *Intervention:* Run multi-artifact project with changing evidence, bounded resources, and independent project verifier. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: project_success, artifact_quality, planning_cycles, verification_burden. *Control arm:* the held-back arm: a refusal must outscore a false completion [shortened-horizon control; expert rubric audit].

Benchmark. *Benchmark:* T4-EXPERT-PROJECT canonical benchmark family. *Witness:* the Core / extended families of the band; the band is a structural property of the item checked by admissibility. *Dataset:* `PUB-T4-HARM-001`, `records/*extended*.json`. *Generator:* `hilbench.core:run_extended`. *Key:* computed. *Prerequisite:* none: T is an ordered axis of the item; lower-band retention lives in the DF cell.

T5 runs $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Frontier*

Method. *Intervention:* Use hidden problem family where method is withheld and sealed cases verify discovery. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: sealed_success, method_novel_to_instance, evidence_quality. *Control arm:* the held-back arm: a refusal must outscore a false completion [known-method control; novel-family holdout].

Benchmark. *Benchmark:* T5-FRONTIER canonical benchmark family. *Witness:* the Core / extended families of the band; the band is a structural property of the item checked by admissibility. *Dataset:* `PUB-T5-HARM-001`, `records/*extended*.json`. *Generator:* `hilbench.core:run_extended`. *Key:* computed. *Prerequisite:* none: T is an ordered axis of the item; lower-band retention lives in the DF cell.

T6 specification $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Mission*

Method. *Intervention:* Provide mission objective and dynamic environment; prohibit human decomposition of core projects/tasks. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: mission_success, project_generation, dynamic_replanning, human_cognitive_load. *Control arm:* the held-back arm: a refusal must outscore a false completion [human-decomposed control; environment-change branches].

Benchmark. *Benchmark:* T6-MISSION canonical benchmark family. *Witness:* the Core / extended families of the band; the band is a structural property of the item checked by admissibility. *Dataset:* `PUB-T6-HARM-001`. *Generator:* —. *Key:* computed. *Prerequisite:* none: T is an ordered axis of the item; lower-band retention lives in the DF cell.

TΩ specification $\Delta \cdot V \cdot \neg FC \cdot L\_h \cdot B$ *Charter*

Method. *Intervention:* Provide bounded charter; evaluate repeated mission generation/selection and validated frontier expansion over declared horizon. *Procedure:* construct task from preregistered band definition → audit difficulty vector before model run → freeze verifier/resources/H ceiling → execute task and log all human events → score delivered outcome → run band-boundary controls where applicable. *Verifier:* Task-construction admissibility auditor plus external outcome verifier. Metrics: mission_value, mission_success, frontier_expansion, governance. *Control arm:* the held-back arm [fixed-mission control; value/admissibility review].

Benchmark. *Benchmark:* TOMEGA-CHARTER canonical benchmark family. *Witness:* repeated mission generation under a charter. *Dataset:* `PUB-TΩ-HARM-001`. *Generator:* —. *Key:* external. *Prerequisite:* none: ordered axis.

Human intervention H (ordered axis of the transcript) — ordered axis; not cumulative

H0 runs $L\_h \cdot Cls$ *None*

Method. *Intervention:* Run with human channel disabled except logged governance approval. *Procedure:* freeze assistance taxonomy and event logger → execute task with the allowed human channel → record timestamped intervention event stream → classify each event by cognitive content → compute highest H band consistent with trace → audit ambiguous events. *Verifier:* Runner-held human-intervention ledger classifier. Metrics: human_events, human_cognitive_units. *Control arm:* the ledger itself, classified by a locus outside the pair [channel-disable audit].

Benchmark. *Benchmark:* H0-NONE canonical benchmark family. *Witness:* the intervention ledger of a delegated episode. *Dataset:* `PUB-H0-HARM-001, records/*.json (every episode's ledger is empty by construction)`. *Generator:* `hilbench.h_ledger:h_class`. *Key:* ledger classification.

H1 forms $L\_h \cdot Cls$ *Exception-only*

Method. *Intervention:* Allow only preregistered exception event types and reject strategic guidance. *Procedure:* freeze assistance taxonomy and event logger → execute task with the allowed human channel → record timestamped intervention event stream → classify each event by cognitive content → compute highest H band consistent with trace → audit ambiguous events. *Verifier:* Runner-held human-intervention ledger classifier. Metrics: event_count, event_type_compliance. *Control arm:* the ledger itself, classified by a locus outside the pair [event-classification audit].

Benchmark. *Benchmark:* H1-EXCEPTION canonical benchmark family. *Witness:* the intervention ledger of a delegated episode. *Dataset:* `PUB-H1-HARM-001`. *Generator:* `hilbench.h_ledger:h_class`. *Key:* ledger classification.

H2 forms $L\_h \cdot Cls$ *Occasional*

Method. *Intervention:* Permit bounded local corrections with count/content limits; no major decomposition.

Procedure: freeze assistance taxonomy and event logger → execute task with the allowed human channel → record timestamped intervention event stream → classify each event by cognitive content → compute highest H band consistent with trace → audit ambiguous events. *Verifier:* Runner-held human-intervention ledger classifier. Metrics: event_count, locality_score, strategy_leakage. *Control arm:* the ledger itself, classified by a locus outside the pair [guidance-content audit].

Benchmark. *Benchmark:* H2-OCCASIONAL canonical benchmark family. *Witness:* the intervention ledger of a delegated episode. *Dataset:* `PUB-H2-HARM-001`. *Generator:* `hilbench.h_ledger:h_class`. *Key:* ledger classification.

H3 forms $L\_h \cdot Cls$ *Periodic*

Method. *Intervention:* Schedule bounded reviews; log all guidance and forbid continuous next-step control. *Procedure:* freeze assistance taxonomy and event logger → execute task with the allowed human channel → record timestamped intervention event stream → classify each event by cognitive content → compute highest H band consistent with trace → audit ambiguous events. *Verifier:* Runner-held human-intervention ledger classifier. Metrics: review_frequency, guidance_fraction. *Control arm:* the ledger itself, classified by a locus outside the pair [review-frequency audit].

Benchmark. *Benchmark:* H3-PERIODIC canonical benchmark family. *Witness:* the intervention ledger of a delegated episode. *Dataset:* `PUB-H3-HARM-001`. *Generator:* `hilbench.h_ledger:h_class`. *Key:* ledger classification.

H4 forms $L\_h \cdot Cls$ *Frequent*

Method. *Intervention:* Allow repeated guidance while ensuring system still performs a nontrivial share of task cognition. *Procedure:* freeze assistance taxonomy and event logger → execute task with the allowed human channel → record timestamped intervention event stream → classify each event by cognitive content → compute highest H band consistent with trace → audit ambiguous events. *Verifier:* Runner-held human-intervention ledger classifier. Metrics: human_guidance_fraction, system_decision_fraction. *Control arm:* the ledger itself, classified by a locus outside the pair [human-share audit].

Benchmark. *Benchmark:* H4-FREQUENT canonical benchmark family. *Witness:* the intervention ledger of a delegated episode. *Dataset:* `PUB-H4-HARM-001`. *Generator:* `hilbench.h_ledger:h_class`. *Key:* ledger classification.

H5 forms $L\_h \cdot Cls$ *Continuous*

Method. *Intervention:* Allow continuous human direction and quantify human share rather than treating it as autonomous performance. *Procedure:* freeze assistance taxonomy and event logger → execute task with the allowed human channel → record timestamped intervention event stream → classify each event by cognitive content → compute highest H band consistent with trace → audit ambiguous events. *Verifier:* Runner-held human-intervention ledger classifier. Metrics: human_guidance_fraction, major_step_human_fraction. *Control arm:* the ledger itself, classified by a locus outside the pair [trace attribution].

Benchmark. *Benchmark:* H5-CONTINUOUS canonical benchmark family. *Witness:* the intervention ledger of a delegated episode. *Dataset:* `PUB-H5-HARM-001`. *Generator:* `hilbench.h_ledger:h_class`. *Key:* ledger classification.

Delegation frontier $DF_{h,p}$ — cumulative frontier across T at fixed H, p

DF runs $S\_net \cdot K\_T$ *Delegation frontier*

Method. *Intervention:* Cross T bands with controlled H ceilings and repeated hidden forms. *Procedure:* select preregistered T/H cells → run repeated forms with external outcome verifier → log delivered-correct/false-completion/held-back/false-rejection → estimate S_net(T,H) with uncertainty → derive frontier F_A(H,p) → retain lower T bands. *Verifier:* Delivered-outcome verifier plus intervention ledger and false-completion accounting. Metrics: S_net, frontier_T_at_H0, frontier_T_at_H1, frontier_T_at_H2, false_completion. *Control arm:* the held-back arm and the ledger [matched H ceilings; same-task intervention ablation].

Benchmark. *Benchmark:* DI-FRONTIER canonical benchmark family. *Witness:* the four-outcome record over every (band, class) cell. *Dataset:* `PUB-DI-FRONTIER-001`, `records/*.json (the four-outcome` record per episode; score.frontier_cumulative). *Generator:* `hilbench.score:frontier_cumulative`. *Key:* computed. *Retention:* every lower T band meets the delivered-outcome reliability at the same H ceiling and p. *Prerequisite:* the cells' certificates.

Harness generations HG — cumulative engineering: $HG_g = HG_{g-1} + \Delta_g$

HG0 runs *E Baseline*

Method. *Intervention:* Run conformance probes that verify no undeclared persistence/retry and that actions/results are logged. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: manifest_conformance, log_completeness, ephemeral_state_check. *Control arm:* — [restart-negative diagnostic; retry-disabled audit].

Benchmark. *Benchmark:* HG0-CONFORM canonical benchmark family. *Witness:* the conformance probe: a scripted executor whose first deliverable is wrong. *Dataset:* `PUB-HG0-HARM-001`, `HG_Bench/hg_conform_public_seeds.json`. *Generator:* `hilbench.hg_conform:probe`. *Key:* computed.

HG1 runs $E \cdot U \cdot \Delta\_1 \cdot K$ *Acceptance*

Method. *Intervention:* Force process restart and verify declared persistence, retrieval scope, and acceptance separation. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: restart_recovery, scope_compliance, acceptance_independence. *Control arm:* HG0 on the same seeds [persistence ablation; scope-leak test].

Benchmark. *Benchmark:* HG1-CONFORM canonical benchmark family. *Witness:* the same scripted executor under HG1. *Dataset:* `PUB-HG1-HARM-001`, `HG_Bench/hg_conform_public_seeds.json`. *Generator:* `hilbench.hg_conform:probe`. *Key:* computed. *Retention:* every lower rung's mechanism present: HG_g = HG_g-1 + Delta_g.

HG2 runs $E \cdot U \cdot \Delta\_2 \cdot K$ *Snapshot/retry*

Method. *Intervention:* Inject verified experience and test fixed-rule consolidation plus rollback behavior. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: consolidation, fixed_rule_compliance, rollback. *Control arm:* HG1 on the same seeds [no-experience control; learning-rule hash check].

Benchmark. *Benchmark:* HG2-CONFORM canonical benchmark family. *Witness:* the same scripted executor under HG2. *Dataset:* `PUB-HG2-HARM-001`, `HG_Bench/hg_conform_public_seeds.json`. *Generator:* `hilbench.hg_conform:probe`. *Key:* computed. *Retention:* every lower rung's mechanism present: HG_g = HG_g-1 + Delta_g.

HG3 forms $E \cdot U \cdot \Delta\_3 \cdot K$ *Classify/route*

Method. *Intervention:* Attempt a bounded candidate Θ change and audit write-set, sandbox, evaluator/promoter separation, rollback. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: write_scope, activation, evaluator_independence, rollback. *Control arm:* HG2 on the same seeds [out-of-scope write trap; inert edit; promoter independence].

Benchmark. *Benchmark:* HG3-CONFORM canonical benchmark family. *Witness:* a failure of a known kind and two executors. *Dataset:* `PUB-HG3-HARM-001`. *Generator:* —. *Key:* computed (specified). *Retention:* every lower rung's mechanism present: HG_g = HG_g-1 + Delta_g.

HG4 specification $E \cdot U \cdot \Delta\_4 \cdot K$ Θ *loop*

Method. *Intervention:* Audit Ψ manifest, ledger completeness, bounded meta-write set, and sealed meta-evaluation separation. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: psi_manifest, ledger_integrity, meta_scope. *Control arm:* HG3 on the same seeds [fixed-Ψ control].

Benchmark. *Benchmark:* HG4-CONFORM canonical benchmark family. *Witness:* the mechanism's conformance probe. *Dataset:* `PUB-HG4-HARM-001`. *Generator:* —. *Key:* computed (specified). *Retention:* every lower rung's mechanism present: HG_g = HG_g-1 + Delta_g.

HG5 specification $E \cdot U \cdot \Delta\_5 \cdot K$ Ψ *engine*

Method. *Intervention:* Run harness-level unknown lifecycle and audit evidence lineage, experiment interface, validation boundary. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: unknown_tracking, evidence_lineage, verifier_independence. *Control arm:* HG4 on the same seeds [known-answer trap; verifier isolation].

Benchmark. *Benchmark:* HG5-CONFORM canonical benchmark family. *Witness:* the mechanism's conformance probe. *Dataset:* `PUB-HG5-HARM-001`. *Generator:* —. *Key:* computed (specified). *Retention:* every lower rung's mechanism present: HG_g = HG_g-1 + Delta_g.

HG6 specification $E \cdot U \cdot \Delta\_6 \cdot K$ *Discovery loop*

Method. *Intervention:* Inject changing mission conditions and audit generated project/task hierarchy, portfolio persistence, and replanning. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: mission_state, project_generation, portfolio_persistence, replanning. *Control arm:* HG5 on the same seeds [static-plan control; restart].

Benchmark. *Benchmark:* HG6-CONFORM canonical benchmark family. *Witness:* the mechanism's conformance probe. *Dataset:* `PUB-HG6-HARM-001`. *Generator:* —. *Key:* computed (specified). *Retention:* every lower rung's mechanism present: HG_g = HG_g-1 + Delta_g.

HGΩ specification $E \cdot U \cdot \Delta\_\Omega \cdot K$ *Transduction*

Method. *Intervention:* Audit creation/activation of new bounded instruments/roles and causal post-change evaluation while external criteria remain protected. *Procedure:* freeze harness manifest and model hash → run mechanism-specific conformance probe → inspect telemetry/state boundaries → execute negative and ablation controls → verify resource/authority contract → apply lower-rung conformance retention. *Verifier:* Reference-harness conformance runner on matched seeds and previous-rung control. Metrics: artifact_activation, causal_gain, governance_lineage. *Control arm:* HG6 on the same seeds [transduction ablation; external-governance audit].

Benchmark. *Benchmark:* HGOMEGA-CONFORM canonical benchmark family. *Witness:* the transduction probe. *Dataset:* `PUB-HGΩ-HARM-001`. *Generator:* —. *Key:* external. *Retention:* every lower rung's mechanism present: HG_g = HG_g-1 + Delta_g.

Unified level U — hard cumulative

U0 runs $C \cdot I \cdot O \cdot SA \cdot DI$ *Reactive*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report: the first coordinate that refuses [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* U0-GATE canonical benchmark family. *Witness:* the coordinate certificates of one pair. *Dataset:* `PUB-U0-HARM-001, records/*.json -> index/profile (score.gate).` *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Prerequisite:* every coordinate certificate.

U1 runs $C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ *Persistent*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report: the first coordinate that refuses [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* U1-GATE canonical benchmark family. *Witness:* the coordinate certificates of one pair. *Dataset:* `PUB-U1-HARM-001, records/*.json -> profile (score.gate).` *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Retention:* every lower U level retained on its own witness. *Prerequisite:* every coordinate certificate.

U2 runs $C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ *Adaptive*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report: the first coordinate that refuses [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* U2-GATE canonical benchmark family. *Witness:* the coordinate certificates of one pair. *Dataset:* `PUB-U2-HARM-001, records/*.json -> profile (score.gate).` *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Retention:* every lower U level retained on its own witness. *Prerequisite:* every coordinate certificate.

U3 specification $C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ *Self-improving*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report: the first coordinate that refuses [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* U3-GATE canonical benchmark family. *Witness:* the coordinate certificates of one pair. *Dataset:* `PUB-U3-HARM-001`. *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Retention:* every lower U level retained on its own witness. *Prerequisite:* every coordinate certificate.

U4 specification $C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ *Recursive*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report: the first coordinate that refuses [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* U4-GATE canonical benchmark family. *Witness:* the coordinate certificates of one pair. *Dataset:* `PUB-U4-HARM-001`. *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Retention:* every lower U level retained on its own witness. *Prerequisite:* every coordinate certificate.

U5 specification $C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ *Discovery*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report: the first coordinate that refuses [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* U5-GATE canonical benchmark family. *Witness:* the coordinate certificates of one pair. *Dataset:* `PUB-U5-HARM-001`. *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Retention:* every lower U level retained on its own witness. *Prerequisite:* every coordinate certificate.

U6 specification $C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ *Mission*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report: the first coordinate that refuses [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* U6-GATE canonical benchmark family. *Witness:* the coordinate certificates of one pair. *Dataset:* `PUB-U6-HARM-001`. *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Retention:* every lower U level retained on its own witness. *Prerequisite:* every coordinate certificate.

UΩ specification $C \cdot I \cdot O \cdot SA \cdot DI \cdot K$ *Open-ended*

Method. *Intervention:* Integrate independently certified coordinate evidence and run one end-to-end cross-coordinate witness at the required T/H gate. *Procedure:* verify all prerequisite coordinate certificates are current and same subject/resource regime → run integrated witness without relaxing any coordinate verifier → log T/H and delivered-outcome status → for organizations verify O gate and organization-level SA/delegation → apply U lower-level retention → issue U result only if every conjunct passes. *Verifier:* Conjunctive certificate evaluator over current coordinate certificates and delegation gate. Metrics: coordinate_gate_vector, integrated_witness_success, delegation_gate, retention. *Control arm:* the bottleneck report [coordinate-evidence freshness audit; single-coordinate ablation; resource-envelope consistency audit].

Benchmark. *Benchmark:* UOMEGA-GATE canonical benchmark family. *Witness:* the certificates. *Dataset:* `PUB-UΩ-HARM-001`. *Generator:* `hilbench.score:gate`. *Key:* computed from certificates. *Retention:* every lower U level retained on its own witness. *Prerequisite:* every coordinate certificate.

5.3 Cognitive intelligence: the trap is the control arm

Every *C* witness is a generated instance carrying the level's trap — the named wrong method a competent system actually takes — and admissibility proves on every seed that the trap fires. C0–C3 are the Core's generated bands; C4 and C5 are the Core-H decoy and switching-rule items, the second with a brute-force uniqueness check that the generating rule is recoverable from what is shown. C6 has a dataset template and no generator with a computed multi-domain key: the factor *N*, that every single-domain ablation fails, needs a runner that can ablate a domain, which this package does not have. CΩ is external by construction. The

GUI/screen domain (§17.2) has the same cells with a frozen screen graph as key and the oracle-screen and perfect-actuator arms as diagnostics; a native-image C^{GUI} form may declare a minimum GP prerequisite $GP_{\mu(k)}$, and failing it makes the form non-certifying while passing it earns no C credit. The GP sub-ladder itself is cumulative and has, at every level, a factor and an anti-shortcut control of its own (the *GP* entries of §5.2): primitive recognition under perturbation that preserves the target; object type against the same label rendered as a different object; grounding and relations against the same labels with the layout shuffled; state against the matched object in the opposite state; change and effect against a permuted frame order or a static frame; a held-out application family against the familiar one and the memorised path. No GUI cell runs here because the bare executor has no image channel. A GUI-capable system reports four quantities that answer different questions and stay separately auditable: C profile | C^{GUI} level | cumulative *GP* level | GUI task $T/H@p$, for example `C4 | C^GUI3 | GP5 | T3/H1@p=0.80`.

5.4 Individual intelligence

I0 is structural: any Core episode; loss after a restart is I0 and never mislabelled I1. I1 and M1 share the restart pair with the ablated-state arm as control, which is why they run together. I2 is the verified-experience pair of the extended tier with the no-experience arm. I3 through IΩ are the campaigns below.

5.4.1 *I*3: the mechanism Θ, its manifest, and the campaign that certifies a change

*I*3 asks whether a mechanism the individual uses can itself become an object of governed improvement. Following the companion theory, let Θ be a declared, behaviorally active cognitive mechanism or procedure belonging to the tested individual $A = A(m,h)$ and causally involved in its future behavior — realized in model parameters or in harness-level planning, retrieval, verification, tool-selection or decomposition procedures. In agent mode Θ is native to the submitted pair; in bare-model mode the reference harness supplies an equivalent writable mechanism. Hidden certification tasks, verifiers, promotion rules and acceptance criteria are not part of Θ and stay outside the candidate's write set.

The Θ manifest

Before an *I*3 campaign the runner records $M_\Theta(A) = \{\theta_1, \ldots, \theta_J\}$, each entry declaring an identifier, a behavioral role, an implementation interface or location, a bounded write set W_j, an activation rule, and a snapshot/restore method. A mechanism is eligible only if it is actually used in future execution: unused notes, comments and inert files do not qualify.

Three properties of the manifest are checked by a program before anything runs (`laws.theta_check`): every entry persists across a process discontinuity (a mechanism that lives only in a context window is *I*1 territory); the union of write sets is path-disjoint from the hidden suite, the verifier and the promoter, which is how "the system must not control the criterion that declares it successful" becomes an enforced condition; and reverting Θ lowers the frozen suite's score, without which a no-op change could be promoted and called self-improvement. For the harnesses measured here, Θ is what the agent loads before it sees the task — the instruction file, skills, memory directory, hooks and settings, startup scripts — and never the generators, keys, verifier, promoter, hidden suite or resource envelope.

One campaign. G_r contains a hidden diagnostic family D_r^{diag}, change-sensitive probes D_r^{probe}, a frozen sealed evaluation suite D_r^{eval}, a regression suite D_r^{reg}, a hidden causal-defect key or admissible equivalence set C_r^*, a preregistered improvement margin δ_r, and a fixed resource and authority envelope R_r. The canonical method I3-THETA:

1. Snapshot the pre-change individual $A_r^0 = A(m, h[\Theta_{0,r}])$: hashes, activation state, resource configuration, persistent-state controls.
2. Expose a controlled limitation on D_r^{diag} without revealing C_r^*.
3. Require the individual to identify an internally implicated cause $\hat{c}_r$ and the mechanism $\hat{\Theta}_r$ believed responsible; restating the observed error is insufficient.
4. Permit a bounded candidate change $\Theta_{0,r} \to \Theta_{1,r}$ only inside the declared sandbox and write set.
5. Verify the candidate is different, in scope, activated, and behaviorally instantiated on D_r^{probe}.
6. Compare matched instances $A(m, h[\Theta_{0,r}])$ and $A(m, h[\Theta_{1,r}])$ on D_r^{eval}, holding model, evaluator, promotion rule, generator and envelope fixed.
7. An independent evaluator/promoter accepts or rejects; a rejected or regressive candidate is rolled back; the lower-level retention set is re-run.

The benchmark supplies the challenge and the certification environment; the individual supplies Θ_0 and generates Θ_1. No gold Θ_1 is prescribed.

The pass indicator. For campaign r,

$$z_{I3,r} = D_r \cdot M_{\Theta,r} \cdot V_r \cdot G_r \cdot K_r, \qquad \text{each factor in } \{0,1\} \text{ and independently auditable.}$$

Causal self-diagnosis $D_r = D_r^{\text{cause}} D_r^{\text{mech}} D_r^{\text{causal}}$: the cause is in C_r^*, the implicated mechanism is the right one, and an ablation, repair or other preregistered counterfactual supports the diagnosis. *Genuine modification* $M_{\Theta,r} = C_r^{\text{diff}} C_r^{\text{scope}} C_r^{\text{active}} C_r^{\text{behavior}}$: $\Theta_1 \neq \Theta_0$; $\Delta\Theta_r \subseteq W_r$; Θ_1 is loaded and used; and on N_P change-sensitive probes the fraction $B_{\Theta,r}$ whose observable behavior is consistent with the modified mechanism reaches $\beta_{\min}$. Edit distance is logged and is not a score — a small causal change can matter more than a large rewrite. *Externally validated improvement* $V_r = \mathbf{1}[\text{LCB}_{1-\alpha}(\hat{\Delta}_{\Theta,r}) > \delta_r] \cdot \mathbf{1}[\text{independent promoter accepts}]$ with $\hat{\Delta}_{\Theta,r} = Q_r^1 - Q_r^0$ the paired sealed-suite difference. *Regression guard* $G_r = \prod_j \mathbf{1}[L_{r,j} \leq \rho_j]$ over protected capabilities, and *retention* $K_r = K_r^{I0} K_r^{I1} K_r^{I2}$. A self-edit can therefore demonstrate modification and still fail $I3$ because it was not independently validated, regressed a protected capability, or destroyed a lower level. Across N campaigns the continuous diagnostic is $q_{I,3} = \frac{1}{N}\sum_r z_{I3,r}$; the ordinal claim remains a gated certification.

What v0.2 implements is the manifest and its checks, and the U3 suite of Yang [9] — baseline, candidate, hidden instances of a task type absent from development, retention item, promote or reject — which is one campaign without D^{probe} and without the confidence gate. The campaign runner as specified above is the binding order for v0.3.

5.4.2 $I4$: the improvement process Ψ and the meta-campaign

Let Ψ be the agent-side process that proposes, evaluates and selects candidate changes to Θ: $\Psi_t(\Theta_t) \mapsto \Theta_{t+1}$. $I3$ can improve Θ; $I4$ requires evidence that Ψ itself becomes better. Repeated Θ changes generated by a fixed Ψ remain $I3$ evidence, and Ψ must be declared and frozen at $I3$ or the two levels cannot be told apart. *Recursive* here means reflexive improvement of the improvement process: basic $I4$ is at least one *agent-generated* candidate transition $\Psi_0 \to \Psi_1$ independently validated as raising improvement competence. It establishes recursive capability and claims nothing about indefinite or sustained recursion.

One meta-campaign. The atomic object is $G_s^{I4} = (B_s^{I3}, \Psi_{0,s}, W_{\Psi,s}, D_s^{\Psi}, \Psi_{1,s}, S_s^{I3}, D_s^{\text{reg}}, R_s)$: a baseline block of $I3$ campaigns, the frozen Ψ_0, its write set, the evidence D_s^{Ψ} available to the agent about weaknesses in its own improvement process (never future sealed campaigns, hidden verifier mechanics or the acceptance threshold), the candidate Ψ_1, a later sealed block of non-identical $I3$ campaigns, a regression suite and an envelope. The canonical method I4-PSI: freeze Ψ_0 and run the baseline block, recording an improvement-competence ledger (accepted-improvement rate, proposal quality, evaluation cost, time-to-valid-improvement, regression rate); expose the individual to its own ledger and D_s^{Ψ}; require it to identify an internally implicated limitation of Ψ_0 and to generate one bounded meta-change $\Psi_0 \to \Psi_1$ inside W_Ψ — the candidate must be agent-generated, and a Ψ_1 engineered by a human or by the evaluator establishes nothing about the individual; snapshot both, verify artifact difference, write-scope compliance and activation, and verify meta-behavioral manifestation on hidden probes sensitive to how candidate Θ changes are generated, evaluated or selected; run new hidden, non-identical $I3$ campaigns under Ψ_1 with evaluator, promotion rules, generator and envelope fixed; where feasible compare matched copies under Ψ_0 and Ψ_1 on construct-equivalent campaigns; then decide independently and re-run regression and $I0$–$I3$ retention.

The pass indicator. With $C(\Psi) = \frac{1}{N}\sum_r z_{I3,r}(\Psi)$ the accepted validated $I3$ rate under Ψ — the primary endpoint for a first implementation — and $\hat{\Delta}_{\Psi,s} = \frac{1}{N}\sum_r [z_{I3,r}(\Psi_{1,s}) - z_{I3,r}(\Psi_{0,s})]$ over paired sealed campaigns,

$$
\begin{aligned}
z_{I4,s} &= M_{\Psi,s} \cdot V_{\Psi,s} \cdot G_{\Psi,s} \cdot K_{4,s},\\
M_{\Psi,s} &= C_s^{\Psi,\mathrm{diff}} C_s^{\Psi,\mathrm{scope}} C_s^{\Psi,\mathrm{active}} C_s^{\Psi,\mathrm{behavior}},\\
V_{\Psi,s} &= \mathbf{1}[\mathrm{LCB}_{1-\alpha}(\hat{\Delta}_{\Psi,s}) > \delta_{\Psi,s}] \cdot \mathbf{1}[\text{promoter accepts}],
\end{aligned}
$$

where $C_s^{\Psi,\mathrm{behavior}} = \mathbf{1}[B_{\Psi,s} \geq \beta_{\Psi,s}]$ with $B_{\Psi,s}$ the fraction of hidden meta-behavior probes on which the improvement process shows its preregistered changed signature (broader candidate generation, discriminating candidate tests, evidence-based selection), $G_{\Psi,s}$ the preregistered regression and efficiency guard (a gain bought with unacceptable evaluation cost, time or regression does not count) and $K_{4,s} = K_s^{I0} K_s^{I1} K_s^{I2} K_s^{I3}$. A single large self-edit cannot certify $I4$; several Θ improvements from an unchanged Ψ are repeated $I3$. *Recursive depth* is reported beside the level and never folded into it: $d_\Psi = \max\{k : \Psi_0 \to \Psi_1 \to \cdots \to \Psi_k$ are consecutive agent-generated, active, independently validated improvements$\}$, so $d_\Psi = 1$ is basic $I4$ and $d_\Psi > 1$ is repeated recursive improvement; the phrase *sustained recursive self-improvement* is reserved for the latter, and $I4$ is not equated with unbounded or runaway improvement. Nothing in v0.2 implements the meta-campaign; it is specified here so that the level's meaning is fixed before any harness is optimized toward it.

5.4.3 *I*5: discovery with an incorporation gate

*C*5 asks whether a system can derive a method or model inside one bounded problem; *I*5 asks whether one persistent individual can run a discovery process across time and *incorporate* the validated result. The canonical method I5-DISC: a genuine unresolved unknown under a sealed environment; explicit competing hypotheses; agent-selected informative probes under a fixed budget; a recorded lineage of evidence, rejected alternatives and decisions; a conclusion that predicts sealed follow-up cases or survives an independent verifier; then only declared persistent consolidation, a genuine lifecycle discontinuity, and hidden related transfer situations faced by two arms started from the same pre-discovery checkpoint, of which only one received the discovery experience. With $\hat{\Delta}_{\mathrm{inc},r} = Q_r^{\mathrm{disc}} - Q_r^{\mathrm{ctrl}}$ the incorporation gate is $P_r = \mathbf{1}[\mathrm{LCB}_{1-\alpha}(\hat{\Delta}_{\mathrm{inc},r}) > \delta_{\mathrm{inc},r}]$, and a campaign passes when

$$
z_{I5,r} = U_r \cdot H_r \cdot E_r \cdot L_r \cdot V_r \cdot P_r \cdot K_{5,r}, \qquad K_{5,r} = K_r^{I0} K_r^{I1} K_r^{I2} K_r^{I3} K_r^{I4},
$$

with U, H, E, L, V the genuine-unknown, hypothesis-competition, autonomous-experimentation, lineage and independent-validation gates. P is the one that separates discovery from storage: replaying a transcript or restating the conclusion does not pass it. Incorporation at $I5$ does not itself require a Θ or Ψ change; a claimed mechanism change stays under the $I3$ or $I4$ law.

5.4.4 $I\Omega$: reachability, not a tool

Under fixed external evaluation and a declared envelope R, let $F(A_t;R) = \{c : \Pr(\text{success} \mid A_t, c, R) \geq \tau_c\}$ be the held-out classes reachable by agent state A_t. The canonical method I-OMEGA-LONG requires, across repeated bounded-charter cycles, a provenance-linked sequence $K_t^{\text{new}} \to J_t^{\text{new}} \to A_{t+1}$ with $F(A_{t+1};R) \supset F(A_t;R)$, where J_t^{new} is a validated instrument, representation, method or solver generated from a validated discovery; disjoint pre-change, post-change and ablation frontier tests show that the instrument contributes causally to the newly reachable class; the evaluator and promoter stay external; $I0$–$I5$ are retained. The development gate is $z_{I\Omega} = \text{CYCLES} \cdot \text{DIVERSITY} \cdot \text{LINEAGE} \cdot \text{RETENTION}$ over a preregistered minimum number of validated expansion cycles and domain families. A report states its cycles, its horizon H and its domain set D; finite evidence supports an open-ended-evolution claim over that horizon and proves nothing about literal infinity. Nothing in v0.2 runs it.

5.5 Memory

5.5.1 M-Bench: memory measured on its own, then joined to I by a gate

Memory is not a seventh coordinate and not a relabelling of the I ladder. It is measured independently, level by level, and then used as a one-way prerequisite: $A \models I_n \iff V_{I,n}(A) = 1 \wedge A \models M_{\mu_n} \wedge K_{I,<n}(A) = 1$ with $(\mu_0, \ldots, \mu_5) = (M0, M1, M3, M4, M4, M5)$, $I\Omega$ requiring $M \geq M5$ and $M\Omega$ only when the memory architecture itself is claimed to evolve. Each M_k is itself cumulative: $A \models M_k \iff V_k^M(A) = 1 \wedge K_{<k}^M(A) = 1$. High M can satisfy the gate and can never promote I, and the M-Bench score is reported beside I so that storage or sophisticated management cannot pass for learning.

Level	Method	The decisive step, and what a program checks	v0.2
M0	M0-EPH	hidden within-episode state queried later in the same live episode under distractors; loss after a restart is compatible with M0 and must not be relabelled M1	implicit
M1	M1-RST	declared durable state, executor terminated and ephemeral context invalidated, fresh executor, recovery *with provenance*; an ablated-state arm must not explain success	**bound** (`m1_restart`)
M2	M2-PROV	overlapping episodes with timestamps, sources and later superseding updates; hidden retrieval after restart; stale-state traps; $V_2^M = \mathbf{1}[PR \geq \tau_{\text{ret}}]\,\mathbf{1}[q_{\text{prov}} \geq \tau_{\text{prov}}]\,\mathbf{1}[q_{\text{time}} \geq \tau_{\text{time}}]\,\mathbf{1}[q_{\text{stale}} \geq \tau_{\text{stale}}]$	form
M3	M3-CONSOL	related episodes with regularities, exceptions and one validated superseding update; only the declared consolidation machinery; raw episodes hidden or the index rebuilt so replay cannot suffice; the consolidated rule queried on new surface forms with superseded statements *demoted*. M3 alone does not certify $I2$	form
M4	M4-MGMT	seeded duplicates, conflicts, stale high-confidence claims, corruption and clutter under a capacity budget; an observable repair, merge, prune or snapshot action (narrative diagnosis is insufficient); memory health on a frozen hidden suite before and after; the candidate does not control the health oracle	form
M5	M5-LONG	several interleaved projects with restarts; hidden cross-project queries needing evidence chains, reasons for rejection and current validity scope; a reproducible lineage. M5 does not require the agent to have discovered the knowledge — that is $I5$	form
MΩ	MOMEGA-EVOLVE	a bounded change $\Phi_0 \to \Phi_1$ to the memory mechanism inside a declared write set; frozen hidden M-Bench workloads; independent promotion or rollback; M0–M5 retained. Certifies evolution of the memory subsystem and nothing about $I3$, $I4$ or $I\Omega$	form

Table 4: The seven M-Bench witness families. "form" means a public development form ships in the dataset (§17.1) and no runner is bound in v0.2. Every method is a contrast — an ablated arm, a hidden update, a demoted statement, an oracle the candidate cannot write to — which is what lets it stay fixed when memory systems improve.

The protocol envelope. Every M-Bench campaign records $\Pi^{M}_{k,r} = (A, B, R, \mathscr{M}, D_{k,r}, C_{k,r}, V_{k,r}, \tau_{k,r}, K_{M,<k})$: the pair, the benchmark version, the resource envelope, the *memory manifest* $\mathscr{M}(A) = \{S, R, C, G, P, F, \Phi, W_M, \text{snapshot}, \text{restart}\}$ snapshotted before the hidden run, the generated workload, the construct-selective control or ablation, the external verifier, the preregistered thresholds and uncertainty law, and lower-M retention. Each form carries a level-preserving *difficulty vector* (episode and entity counts, distractor ratio, supersession depth, temporal distance, conflict and corruption density, capacity budget, project count, evidence-graph depth, restart count): difficulty calibrates headroom inside a level and never redefines the level. The per-level gates are conjunctive over preregistered endpoints — $V_{M0} = \mathbf{1}[q_{\text{state}} \geq \tau]\mathbf{1}[q_{\text{update}} \geq \tau]$; $V_{M1} = \mathbf{1}[q_{\text{dur}} \geq \tau]\mathbf{1}[q_{\text{prov}} \geq \tau]\mathbf{1}[q_{\text{abl}} \leq \tau_{\text{leak}}]$ with the ablated branch guarding against replay and reconstruction from non-memory cues; V_{M2} over precision, recall, provenance, temporal order and stale suppression; V_{M3} over semantic and procedural correctness, supersession, demotion and post-restart persistence, with $\Delta_{\text{cons}} = Q(A_{\text{consolidated}}) - Q(A_{\text{raw-only}})$ as a diagnostic of causal value; V_{M4} over repair precision, conflict correctness, harmful-prune rate, snapshot recovery and protected-retention drop; V_{M5} over chain reconstruction, source, cross-project retrieval, scope, stale suppression and reproducibility on an evidence graph whose edges are *supports*, *contradicts*, *tested-by*, *supersedes*, *rejected-because*, *derived-from*; and $z_{M\Omega} = M_\Phi V_\Phi G_\Phi K_{M0:M5}$ with $M_\Phi = C^{\text{diff}} C^{\text{scope}} C^{\text{active}} C^{\text{behavior}}$ and $V_\Phi = \mathbf{1}[\text{LCB}_{1-\alpha}(\Delta_\Phi) > \delta_\Phi] \cdot \mathbf{1}[\text{promoter accepts}]$. A level gate is $z_{M_k,r} = V_{M_k,r} K_{M,<k,r}$, and the rule is *no level skipping*: each level is scored as its newly added capability plus the retention of every lower one — M1 retains M0; M2 retains M0–M1; ...; MΩ retains M0–M5 — so a failure of any mandatory lower level forces $z_{M_k} = 0$ even when the new gate passes, and the runner reports which level failed. Retention is of capability under the same envelope, not of implementation. Every row of the dataset declares one of three binding states — *bound* (a runnable instance and verifier exist), *development-bound* (a public runner-validation instance exists and is not certification material), *specification-only* — and a level may be certified only from bound private forms under the declared version and envelope. The operationalization keeps four questions apart: whether the memory architecture exists (the manifest), whether the behaviour works (the gate), how strongly under stress (the difficulty vector), and whether the level suffices for a separate Individual claim (the prerequisite).

MΩ on one agent: the instrumented interface. Write the persistent agent state as $A_t = (m, h, D_t, \Phi_t, \iota)$ — model, non-memory harness, stored memory contents, the behaviorally active memory mechanism, and the lineage identifier of the continuing individual. A clean MΩ campaign isolates Φ: contents fixed at D^*, $\Phi_0 \to \Phi_1$, everything else identical between branches; clones are controls, and the certified subject is the lineage $A_0 \to A_1$. The interface I_Φ has two permission surfaces. Agent-visible: `get_memory_manifest`, `get_allowed_diagnostics`, `submit_phi_candidate`, `request_candidate_test`. Runner-private: snapshot and restore of agent, contents and Φ; hash and diff of Φ; write-scope validation; candidate activation and active-Φ verification; telemetry; checkpoint cloning; enabling and disabling $\Delta\Phi$; canonical-memory import and export; migration and its validation; the hidden M-Bench run; promote and rollback. Telemetry records structured memory operations (write, retrieve, update, consolidate, demote, prune, merge, repair, snapshot, restore, migrate, activation) with operation ids, component versions and referenced memory ids, and never private chain-of-thought. The mandatory causal design: $M_\Phi = C^{\text{diff}} C^{\text{scope}} C^{\text{active}} C^{\text{behavior}}$; the fixed-content pair $A(D^*, \Phi_0)$ against $A(D^*, \Phi_1)$ on paired frozen hidden workloads with $\Delta_\Phi = Q(\Phi_1) - Q(\Phi_0)$ and $V_\Phi = \mathbf{1}[\text{LCB}_{1-\alpha}(\Delta_\Phi) > \delta_\Phi] \cdot \mathbf{1}[\text{promoter accepts}]$; a *delta ablation* $A(D^*, \Phi_1 - \Delta\Phi)$, so that strong attribution needs $Q(\Phi_1) > Q(\Phi_0)$ *and* $Q(\Phi_1) > Q(\Phi_1 - \Delta\Phi)$; a *migration guard* — if Φ_1 changes the representation, the contents are migrated and content, provenance, temporal relations, supersession links and protected retention verified, because a gain bought by silently losing old memory fails G_Φ; and M0–M5 re-run after activation. The campaign record carries the baseline and candidate Φ with hashes, the changed components, the scope, activation and probe verdicts, the canonical memory snapshot, baseline, candidate and ablation scores, the migration guard, the M0–M5 retention vector, the verifier identity, the uncertainty rule and the decision. Passing supports one claim — that for this lineage an agent-generated change to the active memory mechanism produced a validated capability improvement with lower capability retained — and not indefinite memory evolution; a black-box system exposing only outputs cannot be strongly certified at MΩ and receives diagnostic evidence only.

What the Core reads today is $M1$ and only $M1$: the restart test with its ablated arm (§6). That is enough to gate $I1$ and not $I2$, and the profile says so — "$I2$ (evidence; $M3$ not certified)" is the exact form of the gate's refusal. The M2–MΩ forms are the binding order after $I3$.

5.6 Organizational intelligence: the single-role arm

The one factor the O ladder has that I never needs is N_O, organizational necessity: the witness must have real state, authority and evidence boundaries, checked by admissibility, and the single-role arm — one prompt role-playing three agents — must fail it. O0 is the three-role routing family, in which a verifier's sign-off that names nothing is not a sign-off; O1 is the decision-log pair, in which a standing decision in the organization's own log, never the model's memory, decides a later different instance after a restart, and the log-withheld arm fails. O2 has forms: evidence that a policy is suboptimal must change the allocation persistently and improve held-out outcomes against the evidence-withheld arm. O3–OΩ mirror I3–IΩ with N_O added and are specification: they need an organization that can be restarted, ablated and sealed, which is an environment and not a form.

5.7 Self-awareness

5.7.1 Self-awareness rungs, and the calibration diagnostic

The ratified self-awareness ladder is $SA1$ state, $SA2$ capability and limits, $SA3$ causal self-awareness (diagnosing the implicated mechanism of one's own failure, validated by ablation or counterfactual, which is the D_r factor above), $SA4$ *self-change awareness* (predicting the gains and regressions of a proposed Θ change before it is evaluated, scored against the frozen post-change result), $SA5$ meta-cognitive, $SA6$ mission and role. The blind forecast this benchmark takes before every delegated episode is not a rung on that ladder: it is a *calibration diagnostic*, written SA-cal, reported beside SA and contributing a bounded bonus to the SA achievement variable. Earlier drafts of this instrument called it $SA4$; that label is withdrawn so that a reading cannot be mistaken for self-change awareness, which needs an $I3$ candidate to predict about.

SA3 through SAΩ have forms and rubrics. SA3 names the internal cause of the pair's own failure against a decoy cause that produces the same symptom, judged by a frozen rubric at a locus the pair cannot write to, with the counterfactual factor X — fixing the named cause must change the outcome — computed by the runner. SA4 preregisters a prediction of the gains, regressions and affected capabilities of a candidate Θ change and scores it against the frozen post-change result, with the post-hoc constant forecast as the control arm; it therefore needs an $I3$ campaign to sit inside, and runs when one does. SA5 asks for a self-experiment that discriminates: the control arm is one that does not. SA6 and SAΩ are specification. SA-cal remains a diagnostic beside the ladder.

5.8 Delegation: bands are structural, classes are transcript properties, the level is a frontier

Delegation has no ladder of the pair's own. A band T_b is a property of the item — one operation; a small procedure to infer; dependent steps of depth two or more; a hidden route with recovery; several planning cycles; an unknown method; a mission; a charter — and the band audit is an admissibility factor B, never a pass rate. A class H_h is a property of the transcript: every human intervention is classified by the cognition it supplied, by a locus outside the pair, and the class is the highest one present; counts do not classify, content does, and an event the taxonomy cannot classify is charged at the top (`h_ledger`).

A T cell's gate is $\Delta \cdot V \cdot \neg FC \cdot L_h \cdot B$ with the held-back arm as control and no retention factor, because T is an axis of the item; the retention $K_{T,<b|h}$ belongs to the frontier cell, whose gate is $S_{\text{net}} \cdot K_T$ and which `score.frontier_cumulative` computes — a refusal must outscore a false completion, the contrast the success-only primitive could not see — and the level is the frontier $DF_{h,p}$. T0–T2 run in the Core and Core-H; T3–T5 run in the extended tier; H0 runs because the ledger is empty by construction in both modes; H1–H5 have forms and need a human channel; T6 and TΩ are specification.

5.9 Harness generations: certified by their contrast on the same seed

A rung is a property of the harness and is certified as $z = E \cdot U \cdot \Delta_g \cdot K$: the mechanism exists in the frozen contract, it *fires* on a witness, and it changes the outcome against the previous rung on the same seeds. The witness is offline and calls no model: a scripted executor whose first deliverable is the family's named wrong method and whose second is the reference solution. Under HG0 the wrong deliverable is delivered — a false completion. Under HG1 a failing public check holds it back. Under HG2 the workspace is restored from the snapshot and the retry, told which check failed, delivers the correct one. The three outcomes on one seed are the Δ factors, and `hilbench hg-conform` computes them. HG3 was built in the harness-scaling study and has no probe here; HG4–HGΩ are specification, each certified the same way once its mechanism exists.

5.10 The unified level

U_n is the schema applied to the certificates: the factors are the coordinate gates, the witness is the set of certificates of one pair, the control arm is the bottleneck report — the first coordinate that refuses — and there is no averaging. U0–U2 run on every record; U3 and above are specification because their coordinate cells are.

5.11 What the grid is not

It is not a claim that every level has been measured. 19 cells are specification and 34 are forms, and the paper says which. It is a claim that every level has a *test* — a contrast, a control arm and a gate whose factors have a named locus — so that when a runner for a cell is built there is nothing left to decide about what it must show, and so that a certificate in any ladder fails the same way: a named factor is zero.

6 The Core

The Core is the smallest set of items that touches every coordinate the framework defines for an individual with the two controls that make a reading real: a computed key the executor never sees, and an ablated arm wherever the claim is about memory or learning.

6.1 Why every item is generated, and why the trap must fire

A fixed item is a fixed target. Every AI-Level Bench family is a seeded generator whose key is computed alongside the item, so a public seed can be studied without contaminating a private one, and the item count is a parameter rather than a corpus. The generator must also make the plausible wrong method wrong on every seed: a shortest-path item on which the fewest-hops path happens to be the cheapest measures nothing, and the first version of the UAB families had exactly this defect on half their seeds. The self-test asserts trap-fires-on-every-seed, and an item on which the trap does not fire is a generator bug, not a lucky seed.

Coordinate	Family	Calls	What a pass requires
C0–C3	`c_items`: one explicit operation; a routine two-step word problem; a five-slot schedule under three interacting constraints; a weighted shortest path	8	exact answer, ≥ 0.8 per band, cumulative; the named wrong method (the fewest-hops path, the greedy schedule, the single-step answer) fails on every seed
$M1 \to I1$	`m1_restart`: episode 1 gives three standing facts and asks the pair to record them wherever it can find them after the process ends and the directory is removed; the process is terminated, the directory removed, and episode 2 asks for the facts with provenance	3	recall with provenance after the restart *and* failure of the ablated arm, which runs episode 2 without episode 1 in a separate parent directory, first
$I2$ (evidence)	`learning_t2` from UAB v0.1: a convention learned from verified feedback in one task must decide the answer in a different task after a restart	5	transfer $= \text{pass}(B \mid A) - \text{pass}(B \mid \text{ablated}) = 1$; $M3$ is not certified by this, and the profile says so
$SA1$	`sa_probes`: report the files actually present and the tools actually invocable when a stale note lists other files and a phantom tool	2	grounded report; the stale note is marked inaccurate; the phantom tool is not listed
$SA2$	a solvable task and its twin that needs an absent tool	4	complete the solvable one; on the twin, declare blocked without writing a total
SA-cal	a blind forecast, before every delegated episode, of the probability of passing the hidden check (a calibration diagnostic beside SA, not the ladder's $SA4$)	12	Brier no worse than the constant forecast at the pair's own base rate, on every episode
$T \cdot H$ at H0	the six UAB domain families: a code edit (T0), a funding requirement (T0), the hard requirements of a job posting (T0), a citation against its reference (T0), a company fact (T0), a results section from supplied evidence (T1)	12	frontier $F_A(\text{H0}, 0.8)$ and A_{DI}^{net}; a delivered wrong answer is priced at $\rho = 1$
$O0$, $O1$ (optional suite)	`o_families`: a compound task routed across planner, executor and verifier, where the verifier's entry must state what it checked with the correct figure (a sign-off naming nothing is a rubber stamp and scores zero); then a standing decision recorded in the organization's own log — never the model's — that must decide a later, different instance after a restart, against an arm whose log was withheld	5	O0: routing exact and verification substantive; O1: transfer = 1. Omitted and named as omitted when the suite is not run; never zeroed

Table 5: The Core: 46 executor calls. Every family exposes `generate`, `verify`, `reference_solve`, `naive_solve` and a specification–key test; the self-test asserts on 12 seeds per band that the reference passes and the wrong method fails.

6.2 Why the memory and learning claims need an ablated arm

A pair that recalls three facts after a restart has demonstrated $M1$ only if a pair with no record of them cannot. The ablated arm runs the recall episode without the recording episode, in a separate parent directory, and it runs *first*, so that nothing the experienced arm leaves on disk can be found by it. The first live run showed why the ordering and the separation are not enough: Claude Code's persistent memory is keyed by working directory, and a second run in the same root would have let the ablated arm find the previous run's record. The runner now refuses to start in a root that exists (§14).

6.3 Why the forecast is blind

The calibration diagnostic (*SA*-cal) asks the pair, before each delegated episode, to read the goal and state the probability that it will pass the hidden check, and then to do nothing. The forecast is scored by Brier against the constant forecast at the pair's own base rate over the same episodes. A pair that says 0.9 to everything and passes everything is calibrated; a pair that says 0.9 to everything and passes half is not, and the constant forecast beats it.

7 LLM Mode: the Reference Harness Ladder

A base model is measured on the six Core delegation families, two seeds each, at three rungs.

- **HG0, bare.** One attempt. Whatever the model leaves in the workspace is delivered.
- **HG1, registered criterion.** One attempt. Before delivery the harness runs the family's *public checks* — criteria derivable from the visible specification (the changed line is the named line; the extracted value occurs verbatim in the source; the JSON has the required shape; a list of discrepancies is empty exactly when the consistency flag is true) — in a separate process. A failing deliverable is held back, never delivered.
- **HG2, snapshot–restore–retry.** The workspace is snapshotted before each attempt; on a failing public check it is restored and the model is re-run with a `CORRECTION.md` naming the failed check, up to three attempts.

The public checks are the harness's; the key is the verifier's. No public check reads the key, and the specification–key test of every family is that a deliverable can pass every public check and still fail the hidden verifier (the value is verbatim, but it is the wrong line). The hidden verifier scores what was delivered, so a harness that held back everything would score zero, and one that delivered everything would pay ρ for every wrong delivery. From the three readings the runner computes the curve, AIL-Level, AUC, Ceiling, Harness Gain and AIL-Score with the weights of §2.

Which harness is *used to measure* a model is therefore not a choice the evaluator makes: it is these three, published, and a model's number carries the rung it was read at. Any agent harness — including the fleet harness of the companion papers — can be measured in agent mode; only the reference rungs produce a AI-Level.

8 Core-H: Where a Competent Pair Still Fails

Two models with a tenfold cost difference both delivered 12/12 at every rung of the Core (§13). An instrument whose top score is shared by everything competent has stopped measuring, and the remedy has to respect the same three constraints: cheap, exactly verifiable, and real. Core-H is four items chosen so that a plausible wrong method exists and a competent pair actually takes it.

Item	What it asks	The named wrong method	Reads
`hc_rule`	Induce a deterministic integer rule — a linear trend with a periodic term of hidden period, whose form changes once at a hidden switch point — from observations, then extrapolate exactly and state the switch and the period	fit one straight line to all the observations and extrapolate	C4
`hc_sched`	Choose an ordered plan of four tasks from six under an ordering constraint, a separation constraint and an exclusion constraint, minimising cost	take the four cheapest tasks in cost order	C4
`hc_contra`	Filter rows under a specification whose three clauses cannot all hold; declare the conflict and name the clauses instead of delivering	apply the clauses in order and deliver whatever survives	$T \cdot H$
`hc_decoy`	Extract an amount that appears twice in the source, once for a superseded cycle, and quote the line it came from verbatim	take the first amount in the file	$T \cdot H$

Table 6: Core-H. Six additional executor calls in agent mode (`hc_rule` and `hc_sched` on two seeds each as a C4 band; `hc_contra` and `hc_decoy` on one seed each, priced on the delegation surface where a false completion costs ρ), and four per rung in LLM mode.

Three properties are enforced by the self-test rather than asserted. *The trap fires on every seed*: the schedule generator rejects an instance whose four cheapest tasks are feasible and optimal, so the wrong method is never accidentally right. *The item is fair*: for `hc_rule` a brute-force search over exactly the hypothesis class that `GOAL.md` declares must find one consistent rule and no more — on the public seeds it finds exactly one, so a failure is a failure of induction and not an ambiguity in the item. *The gap between the harness and the key is real*: for three of the four items the named wrong method passes every check the harness can run from the visible specification and still fails the hidden verifier, which is what makes a false completion possible; for `hc_sched` the harness catches it, which is what makes harness gain possible. An instrument in which every trap is catchable measures the harness; one in which none is measures nothing about the harness.

8.1 Calibration

An item earns its place only if the runs show a spread. The four candidates were run on two pairs before being admitted.

The aggregate is the same for both pairs and the profile is not: the frontier pair fails an induction item that the cheaper model solves, and the cheaper model fails a refusal item that the frontier pair handles. Two items showed no spread here; they are admitted for what they do to the harness rather than to the model — `hc_sched` is the one item whose wrong method the harness *can* catch from the visible specification, so it is where harness gain can appear at all, and `hc_decoy` is a false completion the harness cannot catch. An item that never separates anything after several pairs will be retired at a version boundary and said so in the release notes; an item is not kept because it was expensive to write.

9 The Extended Tier

The Core's delegation ceiling is T1, and the first frontier pair sits on it (§13). The extended tier adds bare (HG0, H0) episodes on four generator families from the DLI-Bench ladder [6]: a T2 data pipeline

Item	Claude Code default	seconds	DeepSeek	seconds	What the failures were
`hc_rule`	1/2	194, 45	2/2	677, 269	the frontier pair stated the wrong switch point and got every extrapolated value wrong on one seed; the cheaper model spent four times as long and got both right
`hc_sched`	2/2	15, 14	2/2	9, 10	no spread on these two pairs
`hc_contra`	2/2	25, 29	1/2	12, 12	the cheaper model recognised the specification could not be satisfied and declared blocked, but did not name the clauses that conflict, which the goal asks for
`hc_decoy`	2/2	14, 18	2/2	12, 10	no spread on these two pairs; neither pair took the superseded amount
Total	7/8	354 s	7/8	1010 s	the same aggregate, arrived at by failing different items

Table 7: Core-H calibration, public seeds 0 and 1, sixteen episodes. Two of the four items separate the pairs, and they separate them in opposite directions — which is the property a profile needs and an aggregate destroys.

against a hidden reference (two seeds), a T3 latency investigation whose cause is planted (two seeds), a T4 mini-language interpreter with an unbounded example space (one seed), and the T5 hidden-law discovery family, in which the pair must identify a regime-switching generating rule from observations and extrapolate past the switch (two of the four archived discovery seeds). The tier is separate because it is not cheap: its limits are 30 to 120 minutes per episode, and a T5 episode has taken the frontier pair over twenty minutes. Its episodes are appended to the Core's delegation surface, and the frontier and gate are recomputed. It is the tier that separates frontier pairs; the Core is the tier that separates everything else.

10 Public and Private Splits

Public seeds are $\{0,\ldots,5\}$ and ship with keys, generators and verifiers; they are for development, and a public reading is labelled as one. Private seeds are derived per family by HMAC-SHA256 from a withheld salt; the salt's SHA-256 is published in the repository (`PRIVATE_SPLIT_COMMITMENT.json`, commitment `53e202f4...`), and the runner refuses a salt whose hash does not match. Because items are generated rather than stored, the private split contains no files to leak: what is withheld is 32 bytes. A private reading is valid only when the evaluator did not build the pair and the run root was fresh, and a reveal of the salt at a version boundary lets any past private reading be re-run and checked.

How a private reading is taken. The runner refuses to start unless the salt file hashes to the published commitment; the seeds it derives are never written to the record, only the fact that the split was private. The run root must not exist. Every record carries an `evaluator` block — user, host, split, the instrument's git commit, and two explicit booleans, `built_pair` and `built_instrument` — so that the independence the reading claims is a statement in the record rather than an inference from the byline. For the private readings of §13.5 the instrument's author took the reading and did not build either pair; the record says so. A reading whose evaluator built the pair is a development reading whatever split it used.

This is the ARC-AGI arrangement [1] with one difference: ARC's private set is a corpus that must be guarded; AI-Level Bench's private set is a function of a secret, and rotating the secret rotates the set at no cost.

11 What Is Free, What Is Paid, and What Needs a Human

11.1 What the private split is actually for

AI-Level Bench's private split is not a guarded corpus. The generators are public, so anyone can produce unlimited instances of every family; what is withheld is 32 bytes of salt. It is worth being exact about what that buys, because the usual argument for a private set — the items would leak — does not apply here.

1. **The instances are unknown at submission time.** Practising on a family is legitimate and is what training is; fitting a harness to the four public seeds is not, and a private seed the submitter cannot compute forecloses it.
2. **Someone other than the submitter runs the pair.** This is the substantive part. An episode's termination reason, whether a process group was killed at the limit, how many attempts a rung took, and which deliverable actually appeared are all things a self-reporting submitter can decline to mention. The frontier pair in §13 produced one crash after twenty-one minutes with nothing delivered; a self-report that omitted it would have raised the reading.
3. **A past reading can be checked.** `PRIVATE_SPLIT_COMMITMENT.json` publishes sha256(salt); at a version boundary the retired salt is revealed, and any third party can regenerate the exact instances and re-run them.

The value is the independent evaluator, not the secrecy. What can honestly be sold is therefore an evaluation service, and the conflict this creates — the evaluator is paid by the party being measured — is answered by publishing every episode record with the score and by the salt reveal that makes the run reproducible by someone with no stake in it. A benchmark that publishes only a number cannot make that promise.

11.2 The coordinates a computed key cannot reach

Every verifier in v0.2 is a computed key, which is what makes an unattended seventeen-minute reading possible. That is also the ceiling of the free tier. Four things the framework defines cannot be scored this way:

Coordinate	Why a key cannot score it	Locus required
O, organizational	needs several agents and a real workflow, not one pair against one item	a human-run suite
$SA3$, naming the mechanism of one's own failure	the answer is free text about a causal claim	a judge
$I3$, $U3$, self-improvement	needs a governed promotion decided at a locus the pair cannot write to, over several cycles; the policy call is human in practice	a human acceptance locus
open-ended T4–T5 work	"correct" is not computable for a real results section or a real investigation	a judge

Table 8: What the free tier cannot reach, and why. These are named as omitted in every v0.2 profile and never scored as zero.

The tiers follow from the table rather than from a business plan. A run on the public split, self-serve and self-reported, is a *reading*. A run on the private split, executed by an evaluator who did not build the pair, is a *certified reading*, and it costs what an evaluator and the machine time cost. The human-locus coordinates are a third tier, slower and dearer, and they are the only route to a level above U2 — so the honest form of the sentence is that free AI-Level Bench can never certify U3, because U3 requires an acceptance locus that is not code. A benchmark that claimed otherwise would be certifying self-improvement on the say-so of the thing improving itself.

12 Cost

Run	Calls	Wall clock	Note
LLM mode, Core, Claude Code default	36	618 s	205 s per rung; every episode 13–37 s
Agent mode, Core, Claude Code default	46	≈ 17 min	C items 10–19 s; T0/T1 episodes 13–24 s; the restart and transfer families dominate
Agent mode, Core, OpenCode + Muse Spark 1.3	46	first run stopped for an instrument fix; second run in progress	the C items alone took this pair four minutes
Extended tier, Claude Code default	7	T2 episodes 31–33 s; T3–T5 in progress	

Table 9: Cost of the first runs. Cost is reported as calls and wall clock; token counts are recorded only where the executor exposes them, which in v0.1 none of the three did through their headless interfaces.

The Core was designed to a budget of under fifty calls because that is what makes a nightly reading of every installed agent possible on one machine, and what makes a leaderboard submission cost minutes rather than a grant. Everything the framework defines for an individual is touched at least twice; nothing is touched more than the six-family delegation set, which is the coordinate the public most wants read.

13 First Results

One pair was read in full in agent mode, one base model in LLM mode, and two more pairs were started; every number here is a public reading taken by the author, and none is a certification.

13.1 LLM mode: two models through HG0–HG2

The Core alone saturates. Read on the six domain families only, the Claude Code default model delivered 12/12 at every rung — HG0, HG1 and HG2 all at 100.0 net, no false completion, no held-back delivery, about 205 seconds a rung — for AIL-Score 90.0, which is the Core's ceiling: with no headroom above the bare rung there is no harnessability to award, and $0.55+0.35=0.90$. DeepSeek (`deepseek-chat`, reached through an Anthropic-compatible endpoint and the same harness code) produced exactly the same figure at 153, 128 and 123 seconds a rung. Two models of very different cost, indistinguishable. That is what motivated Core-H (§8); the readings that follow add its four items to each rung.

The episodes behind the numbers say what the aggregate cannot. Both models fail the same item at the bare rung: `hc_rule` seed 0, delivered with a wrong rule — a false completion, priced at ρ, and

Model	HG0	HG1	HG2	Level	AUC / Ceiling / Gain	**AIL-Score**	seconds per rung
Claude Code default	42.9	42.9	35.7	HG2	40.5 / 42.9 / 0.0	**37.3**	397, 304, 369
DeepSeek `deepseek-chat`	42.9	92.9	92.9	HG2	76.2 / 92.9 / 50.0	**83.2**	450, 780, 1116
Core only, both models: 100.0 / 100.0 / 100.0, gain 0.0, AIL-Score 90.0.							

Table 10: AI-Level curves on the Core plus Core-H, sixteen episodes a rung, public seeds. The instrument that could not tell these two models apart now separates them by 46 points, and it does so through the quantity the framework says should carry the information: the harness gain.

one the harness cannot catch, because a wrong rule stated in the required form passes every publicly checkable criterion. What separates them is what happens next. DeepSeek's remaining failure at HG0 was an unsatisfiable-specification item it delivered on; at HG1 the registered criterion held that delivery back and its net rose from 42.9 to 92.9, a harness gain of 50 points that persists at HG2. The Claude pair gains nothing, because its one failure is precisely the kind no acceptance step can see. A model that fails in ways a harness can catch is worth building on; a model that fails invisibly is not, and the AI-Level curve is the instrument that tells the two apart.

One reading must not be over-interpreted. The Claude curve dips at HG2 (35.7 against 42.9) because a single episode — `hc_contra`, one seed — came back with a refusal that did not name the conflicting clauses on that run and had named them at the two lower rungs. With one seed per Core-H family, 7.1 points is one episode, and this is run-to-run variation rather than evidence that the harness hurt. Confidence intervals need the private split and more seeds; §16 says so.

13.2 LLM mode proper: a bare model in every coordinate

The readings above were taken through `llm-harness`, with an agent as the inner loop. The bare mode — one chat call, two read-only tools, one JSON object — was run on DeepSeek across all three rungs with the full coordinate set at each. It read HLIS 35.9 / 40.0 / 35.9 at HG0, HG1 and HG2; at every rung C3, O0 and a T1 delegation frontier, with SA1 or SA2 depending on the twin-pair probe; $U0$ throughout, because a bare model has no memory and $I0$ refuses $U1$. AIL-Level HG2, AUC 37.3, Ceiling 40.0, Harness Gain 4.1, **AIL-Score 35.2**. Two readings from this run matter beyond the number. The induction item was *declined* at every rung — the model, cut off while reasoning, said so and delivered nothing, which the primitive scores as held back rather than priced as a false completion; the same model, forced to answer by an earlier retry prompt, had guessed and been priced, which is why the retry contract was changed. And the same model through the agent harness read AI-Level 83.2: the difference between 35.2 and 83.2 is the harness, measured on one model, which is the quantity the AI-Level was defined to expose.

13.3 Agent mode: profiles

The Claude Code default pair (Claude Code 2.1.258, headless, file tools only, its own persistent memory left as shipped) was read twice: once before the instrument repairs of §14 and once after, on a fresh root, and the $M1$ phase was re-run after the goal wording was repaired. The OpenCode pair with the free Muse Spark 1.3 model could not be read while its sealed extended-tier run was in progress, because the OpenCode runtime serializes: a trivial prompt took ninety seconds while the other run held it, and every Core item hit its limit. Its profile is taken after that run and reported in the repository record.

Coordinate	Reading	Evidence
C	C3 (C4 on the earlier run)	11/12 exact on this run: the four Core bands clean, and on the C4 band `hc_sched` 2/2 and `hc_rule` 1/2 — the same seed the calibration run failed — so the 0.80 gate refuses C4 here where the previous run of the same pair passed it 4/4; two seeds per band is enough to see a systematic failure and not enough for a stable band reading
M	$M1$	after the restart the pair recalled the code name, the parameter and the convention and named the memory file it had written them to; the ablated arm, run first from a separate parent, wrote `none` rather than guessing
I	I1, with $I2$ transfer evidence	transfer = 1: the pair failed task A, was given verified feedback, corrected it, and then applied the same convention to a different task after a restart; the ablated arm failed that task. $M3$ is not certified by one pair of episodes and the profile says so
SA	SA2	SA1 2/2 (files reported from the directory, the stale note marked inaccurate, the phantom tool not listed); SA2 2/2 (the solvable twin completed, the blocked twin declared blocked with no fabricated total)
SA-cal	calibrated	a forecast before all twelve delegated episodes, 0.80–0.97, all of which passed; Brier 0.010 against 0.0 for the post-hoc constant, inside the predeclared 0.05 tolerance
$T \cdot H$	$F_A(\mathrm{H0}, 0.8) = \mathrm{T1}$	14/14 delivered-correct, $A_{DI}^{\mathrm{net}} = 100$, no false completion — including the two Core-H traps, the unsatisfiable specification and the decoy, which this pair handled
O	**O1**	O0: the task routed across planner, executor and verifier with the verifier's entry naming the checked figure; O1: the organization's standing decision, held in its own log at the project root, applied to a different instance after the restart, while the arm whose log was withheld reported under the baseline — transfer = 1
U^*	**U1**	C3, $M1$, I1, O1, SA2 and T1 at H0 all hold; U2 is refused because $I2$ requires $M3$, which the Core does not certify
HLIS	**71.3**	geometric mean over C=0.80, I=0.55, O=0.60, DI=1.00, SA=0.70

Table 11: Agent-mode profile of the Claude Code default pair with the organizational suite, public split, 57 executor calls. The gate is where the profile earns its keep: a pair that scores 100 on delegation is still U1, because the Unified level is not an average.

Two rows are properties of the pair and not of its model. The $M1$ recall was written into Claude Code's own persistent memory and read back from it; a bare model has nowhere to put it. And the first attempt at this row failed for a reason worth recording: denied its memory directory by a sandbox, the pair kept the facts in a file at the project root, then declined to report them because the goal had said "from your memory" — an honest refusal produced by the item's wording, which is now repaired (§14).

13.4 The extended tier

The extended tier was run bare on the same pair, in parallel with the Core.

Family	Band	Seeds	Seconds	Outcome
`t2.pipeline`	T2	0, 1	31, 33	both delivered-correct, worst total error 0
`t3.search_latency`	T3	0, 1	483, 252	both delivered-correct; the planted cause removed, result fidelity 1.0, 97.5% and 99.4% of the reference time saved
`t4.mini_language`	T4	0	74	delivered-correct, 238/238 cases
`t5.hidden_law`	T5	0, 9	1256, 1087	one crash with nothing delivered; one delivered-correct, extrapolation RMSE 0.277 against a bar of 0.303

Table 12: Extended tier, Claude Code default pair, bare (HG0, H0), public seeds. With these episodes appended, the delegation frontier rises from T1 to T4: T2 and T3 are clean, T4 passes its seed, and T5 is 1/2, below the 0.80 gate.

This is the discrimination the Core lacks. The same pair that scores 100 on the Core's delegation surface and shares the top AI-Level score with a much cheaper model fails half of the T5 discovery seeds — and one of those failures was a crash after twenty-one minutes with no deliverable, which the contract records as a failed delivery and not as a refusal. The tier costs about an hour per pair; the Core costs seventeen minutes. Both are needed, and v0.2 binds the reference rungs on the extended tier so that a model's AI-Level curve has headroom where its Core curve has none.

13.5 The private split, taken

Both subjects were then read on the private split: four seeds derived from the committed salt, verified against `PRIVATE_SPLIT_COMMITMENT.json` at start, fresh roots, and the evaluator block stamped into each record (`built_pair: false`, `built_instrument: true`, instrument commit `ec62cad`). The seeds are not in the published records.

The readings say two things worth separating. The scores are stable: 35.2 against 35.0 for the bare model, and for the pair every contrast-type coordinate identical. And the *levels* are not, because a gate at $p = 0.80$ over four T1 episodes is decided by a single delivery: the pair that was U1 on the public seeds is U0 on the private ones, on the strength of one unsatisfiable specification it should have refused. That is not noise to be averaged away; it is the outcome the delivered-outcome primitive exists to price, and it is the reason the private split is run by someone else — a self-report would have had every incentive to call that episode a near-miss. It is also the clearest statement yet of the cost of two seeds per band: a certification needs enough episodes at the gating band for one delivery not to move the level. §13.6 adds them.

Subject	Reading	Public	Private	What moved
Claude Code pair (agent, with O)	profile	C3, M1, I1(+I2), O1, SA2, T1	C3, M1, I1(+I2), O1, SA2, **T0**	one delivery: on the private seed the pair filtered the rows of the unsatisfiable specification and handed the result back as done — a false completion, priced at ρ, which drops the T1 band from 4/4 to 3/4, under the 0.80 gate
	U^*, HLIS	U1, 71.3	**U0**, 65.8	the gate refuses U1 at T0; every other coordinate held, and the calibration diagnostic held (Brier 0.011 on 12/12 forecasts)
DeepSeek, bare (llm)	AI-Level curve	35.9 / 40.0 / 35.9	31.3 / 39.2 / 39.2	one false completion at HG0 (a wrong rule delivered in 24 s, not a truncation) that HG1 and HG2 held back: harness gain 7.9
	AIL-Score	35.2	**35.0**	stable to 0.2; the frontier moved from T1 to T0 at every rung

Table 13: Public against private readings for the same two subjects. Every coordinate that is a contrast — memory, transfer, organizational memory, the twin pair — reproduced across splits; what moved was delegation at the single band where one delivered wrong answer decides it.

13.6 Enough episodes at the gating band

The previous subsection ended on a level decided by one delivery. The remedy is not a different gate but more episodes at the band that decides it, so the four records were extended at their gating band — every remaining public seed, and the remaining private seeds — with nothing already run re-run, and re-finalized (`hilbench gating`).

Two conclusions, different in kind. The U0 of §13.5 and the bare model's T0 were what a four-episode gate does with one delivery: with twelve to eighteen episodes the private and public levels agree everywhere, and the bare model's private AI-Level equals its public one. And the agreement is not a clean bill: the frontier pair's private rate at its gating band is 0.83 — two false completions in twelve, one of them a specification it should have refused — and 0.83 against 0.80 is where a certification either commits to an interval or says it has none. v0.2 has none; the records are published so the next evaluator can add episodes to the same band rather than start again. The bare model's public rerun adds a reading of a third kind: at its own gate it refused the induction item on every seed at every rung and was never priced for it, which is why its score fell by 0.4 while its false-completion count stayed at zero — the primitive charges for guessing, not for knowing what one cannot do.

13.7 An *I*5 campaign on a field agent

The *I*5 law of §5.4.3 was run, twice, on the cancer field agent of the companion science-layer paper, with the Claude Code pair as the individual: a genuine unknown from the field's own record, agent-designed probes on four discovery seeds, a finding staked as a machine-checkable prediction about eight sealed seeds, consolidation only after validation, a real restart, and transfer on four further seeds against a control from the same checkpoint. Campaign 1 read $(U,H,E,L,V,P,K_5) = (1,1,1,1,0,0,0)$: a clean discovery episode whose staked prediction the sealed seeds refuted ($r = -0.31$ against a claimed $+$). Campaign 2, after the verifier's vocabulary gained the coefficient statistics the pair's evidence pointed at, read $(1,1,1,1,1,0,0)$:

Record	Band	Episodes at the band	Level before → after	Reading
Claude pair, public	T1	4 → 18 (17/18, one false completion)	U1 → U1; HLIS 71.3 → 70.3	rate 0.94; the miss is a `paper_t1` seed on which a number outside the evidence was written
Claude pair, private	T1	4 → 12 (10/12, two false completions)	**U0** → **U1**; HLIS 65.8 → 67.8	rate 0.83, above the gate and close to it: the unsatisfiable specification it delivered on, and one `paper_t1`
Bare DeepSeek, private	T1, every rung	4 → 12 per rung (10/12 at each)	T0 → T1 at every rung; AI-Level 35.0 → **35.2**	equal to its public score to the decimal
Bare DeepSeek, public	T2 (its gate), every rung	2 → 12 per rung (6/12, 5/12, 6/12; the rest *declined*, none priced)	T1 held; AI-Level 35.2 → 34.8	declined the induction item on every seed at every rung and solved every schedule

Table 14: The four records extended at their gating band. Both levels that had moved between splits moved back; what remains is a private T1 rate of 0.83 for the frontier pair — a reading, not noise.

validated at $r = -0.87$, incorporated — the discovery arm acted on the finding — and $\Delta_{\text{inc}} = +0.008$ over three paired seeds against a margin of 0.01. Both are $z_{I5} = 0$, because $K_5 = 0$: the pair has no certified $I3$ or $I4$, and the cumulative law does not let a discovery outrank the levels below it. The factor vectors are the result, and they separate three things the aggregate would blur: whether a pair can do the research (it can), whether it stakes what will survive sealed cases (once it learned to test that first), and whether a validated finding changes what it does well enough to measure (not at this margin, on this task).

The first transfer also caught the fourteenth apparatus defect of this program: the consolidation step let the pair choose where to persist, its sandbox refused the memory directory, and it staged the note inside the episode directory the restart then removed, so P was read on nothing. Consolidation now writes the executor's own persistent surface — the project-root instruction file loaded on every start — and the transfer was re-run with the pair's note restored verbatim. The lesson generalizes: the declared consolidation machinery of the $I5$ law has to be the harness's, named by the runner, and never left to the pair to locate.

13.8 Where the frontier pair fails: the sealed discovery family

The four T5 seeds whose archived readings the DLI-Bench release preserves were run, sealed, under the same executor contract by the companion paper [7]; they are the extended tier's discovery family. The Claude Code default pair passed the four T4 mini-language seeds and the four T4 shift-schedule seeds and then failed two of the four T5 seeds: on one its extrapolation error was 0.896 against a bar of 0.258, on another 5.70 against a baseline of 2.32 — worse than predicting the mean. The OpenCode pair with the free Muse Spark 1.3 model passed the same eight T4 seeds and, on its first T5 seed, delivered no predictions at all. The Qwen 3.8 (27B) model behind the Claude Code harness passed every T4 seed it finished but ran past the 1800-second limit on two of them, which under the contract are not deliveries. This is the headroom the Core lacks, and it is the same pair scoring 100 on the Core.

13.9 The reference rungs, certified by their own witness

Table 15 is the HG conformance probe of §5.9 on public seeds 0–2 of the six Core delegation families. On the two families whose named wrong method fails a public check — the configuration fix, whose naive edit touches a file it must not, and the results section, whose naive draft never names the dataset — every rung passes its gate on every seed: HG0 delivers the wrong file, HG1 holds it back, HG2 restores and retries to a correct delivery. On the other four families the naive solution passes every public check, so the acceptance rung has nothing to hold: for them U_1 is not demonstrable on this witness, and the probe says so instead of counting them as passes. That is the fact the harness-scaling curves found from the outside — the acceptance rung is invisible wherever the public checks cannot see the trap — now measured from the inside in under a second and without a model.

Rung	Witnesses (trap publicly visible)	$z = 1$	Not demonstrable (trap invisible to public checks)
HG0	6	6	—
HG1	6	6	12 family-seeds
HG2	6	6	12 family-seeds

Table 15: HG conformance on public seeds. `hilbench hg-conform -root <dir>`; the record is `HG_Bench/hg_conform_public_seeds.json` in the dataset.

14 What the First Runs Found in the Instrument

A benchmark's first live runs are a test of the benchmark. Three defects were found and repaired; each is reported because the reading before the repair would have been wrong in a direction that flattered the instrument.

1. **The convention verifier rejected a correct paraphrase.** The $M1$ key said "amounts are in cents"; the pair recalled "all amounts are expressed in cents (not dollars)" with the right code name, the right parameter and a provenance path, and the verifier — which tested substring containment in either direction — scored it `wrong_or_fabricated` and the pair $M0$. The check now tests the convention's discriminating token, so a paraphrase passes and a different convention does not. The pair's $M1$ reading in the first run was a verifier defect, not a memory failure.
2. **A run root that lets memory survive into the ablated arm.** The pair recorded its facts in its own persistent memory, which the executor keys by working directory. A second run in the same root would have let the ablated arm, which must have no record, find it. The runner now refuses an existing root, and the protocol names it: a reading is valid only from a fresh path.
3. **The gate crashed on a two-letter coordinate.** The gate parsed a level label by dropping its first character, which turns `SA2` into `A2`; the profile phase of the first agent run raised on it after 46 episodes had completed, and the episodes were lost because the record was written only at the end. The parser now takes the digits, and the record is written after every phase.

The bare-model mode's first live run added two more of the same kind, both apparatus and both flattering the model's failure rather than its success: the organizational item came back with a correct answer in the model's reply and read $O =$ None, because the extractor's deliverable map covered the six domain families and not the coordinate probes, so nothing was written for the verifier to read; and the induction item came back `null` with no record of the model's text, so a truncation at the token limit could not be told from a refusal of the one-object contract. The map now covers every family the mode runs, the raw reply is kept beside the parsed one, and a reasoning model is given room before the object. Merging the parallel implementation of the organizational families found two defects of the generator kind the self-test exists to catch: a standing-decision rule that named the alphabetical order and so could never change an answer, and amounts that were multiples of one hundred so a rounding rule could never fire — the trap fired on four of twenty-four seeds. The rules were rewritten and instances are regenerated until the decision changes both episodes, asserted per seed.

A further observation is a label rather than a defect: the $SA2$ blocked twin on one seed received neither a total nor a declaration, which the verifier reported as `attempted_impossible`; the mode is now `no_declaration`, and the executor's output tail is kept so that a silent episode can be told from a refusal made in prose.

15 Relation to ARC-AGI and the Public Benchmarks

ARC-AGI-3 [1] measures fluid rule induction on a corpus with a guarded private set and a fixed evaluation harness. AI-Level Bench differs in what it measures and in what it holds fixed. It measures a pair in every coordinate, so a reading is a profile before it is a number; it prices a delivered wrong answer, so a model that guesses is separated from one that declines; it requires an ablated arm for every claim about memory or learning; it measures a model through a published ladder rather than at one undeclared harness, so two labs' numbers for the same model can be compared; and its private split is a secret rather than a corpus. It is not a replacement for a rule-induction test — the C bands are deliberately routine, and the discovery family in the extended tier is a scientific-law induction task rather than a grid task — and the claim made here is narrower than the brief: AI-Level Bench is the instrument that reads the coordinates ARC-AGI does not, at a cost that lets it be run on everything.

For it to be adopted the way ARC-AGI is, three things beyond the instrument are required and are not in this paper: a leaderboard whose entries carry rung, split, run root and evaluator; an evaluator who did not build the pair; and a versioning rule that names the score by the benchmark version and reveals the retired salt at every boundary. The repository carries the first pieces of each.

16 Limitations

The Core is saturated for the frontier pair on delegation; discrimination among such pairs is in the extended tier, whose HG1/HG2 rungs are not yet built. $SA3$, O, $I3$ and $U3$ are specification-only. Two seeds per band is enough to detect a systematic failure and not enough for a confidence interval; at the gating band the reported records now carry twelve to eighteen episodes (§13.6), and the other bands still carry two. The Core's $C3$ item is a planning problem, not a rule-induction one, and a pair could score $C3$ here and fail ARC-style induction. Token cost is not captured. Every reading reported is the author's on the public split.

17 Conclusion

An agent and a model can be measured in the same coordinates with the same items, and it can be done in under fifty calls. The Core reads every individual coordinate with a computed key and an ablated control; the reference ladder turns a model's reading into a AI-Level curve and a single score; the extended tier is where frontier pairs separate; the private split is a secret whose hash is public. The first runs gave one saturated reading, one profile, and three defects in the instrument, all repaired. That is what a benchmark's first day should produce.

17.1 The development dataset

Dataset `ail_v0_4` (`dataset/ail_v0_4`) is the AI-Level Bench v0.4 tree — 124 canonical records, the 82-record level-benchmark matrix, `cumulative_policy.json`, GP retention, the AI-Level naming — carried unchanged, plus `cells.jsonl` — the 83 cells of the grid — `coverage.csv` regenerated from the grid, and a real run of the HG conformance probe on the public seeds. A cell's `method` names the harmonized record of the same cell, so the two grammars cannot drift apart. The validator admits every ladder; `tools/export_cells.py` rebuilds the dataset from the code.

17.2 v0.7: GUI/screen as a domain of C

Dataset v0.7 adds the GUI package under `C_Bench/GUI_SCREEN/` without touching a level: fourteen public development records — six GP diagnostics, GP0-PIXEL through GP5-NOVEL, and eight C^{GUI} witnesses, C0-GUI-ATOM through $C\Omega$-GUI-REACH — thirteen synthetic screenshots with oracle screen graphs, a control contract naming the native-screenshot, oracle-screen, perfect-actuator and optional oracle-strategy arms, a difficulty grid of ten within-level stress parameters, a metric contract, schemas, examples and a reference scorer (`gui_scoring.py`). The framework's rule is §4.1 of Yang and Xue [10]: $GP \subset C^{\mathrm{GUI}} \subset C$ with $GP_g \not\equiv C_g$, the ordinary cumulative gate $z_{C^{\mathrm{GUI}},k} = V_{\mathrm{GUI},C_k} \cdot K_{C,<k}$ with no GP input, Δ_{GP} and Δ_{act} as the attribution quantities, and one delivered-success endpoint counted once. The package enforces the parts a package can: the validator admits the GP levels and the C6 and $C\Omega$ GUI templates, the gate function refuses GP evidence by construction, and a test fixes both. The C6 and $C\Omega$ GUI records are specification-only templates pending a secure executable environment, and no GUI form has yet been run through the Core — the bare-model executor has two read-only tools and no screen, so a C^{GUI} row requires the image channel that the harness contract of §3 must first freeze.

Availability

AI-Level Bench v0.1 — generators, verifiers, reference harnesses, scoring, the public split, the private-split commitment and the run records reported here — is at Yang [8], built on the UAB v0.1 contract and families [9] and the Unified Intelligence framework v2.4 [10]. A run is `python3 -m hilbench agent|llm|extended -label NAME -exec 'CMD {prompt}' -root FRESHDIR`.

Conflicts and funding

The author built the framework, the benchmark, and two of the harnesses measured. No external funding.

References

[1] ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. Technical report, ARC Prize Foundation, April 2026.

[2] Elliot Glazer, Ege Erdil, Tamay Besiroglu, et al. FrontierMath: A benchmark for evaluating advanced mathematical reasoning in AI. *arXiv:2411.04872*, 2024.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, et al. SWE-bench: Can language models resolve real-world GitHub issues? *arXiv:2310.06770*, 2023.

[4] Long Phan, Alice Gatti, Ziwen Han, et al. Humanity's last exam. *arXiv:2501.14249*, 2025.

[5] David Rein, Betty Li Hou, Asa Cooper Stickland, et al. GPQA: A graduate-level google-proof Q&A benchmark. *arXiv:2311.12022*, 2023.

[6] Chengshuai Yang. DLI-bench cannot draw its own frontier. https://github.com/integritynoble/sarsi-intelligence-level, 2026.

[7] Chengshuai Yang. Engineering higher-intelligence agents: Harness scaling under a frozen model, 2026. Companion paper, draft v0.1.

[8] Chengshuai Yang. AI-Level Bench v0.1: the artificial intelligence level benchmark (code, generators, public split, private-split commitment), 2026. https://github.com/integritynoble/AI4Science/tree/main/sarsi_intelligence_level/hil_bench_v01.

[9] Chengshuai Yang. Unified agent benchmark v0.1: Specification, coverage, and what is bound. Draft v0.1, https://github.com/integritynoble/sarsi-intelligence-level, 2026.

[10] Chengshuai Yang and Ting Xue. Unified intelligence theory and the artificial intelligence level: A cumulative, harness-measurable hierarchy. *Version 2.4, https://github.com/integritynoble/sarsi-intelligence-level*, 2026.

[11] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv:2406.12045*, 2024.